
Development and implementation of an ECM:

A generalist approach to user-AI interaction.

School: Escuela de Ingenieria de Fuenlabrada.

Degree: Robotics Software Engineering.

Author: Sebastián Mayorquín Posada.

Tutor: Julio Vega.

June 10, 2025

LICENSE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

ACKNOWLEDGEMENTS

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

ABSTRACT

[En proceso] Los ultimos lanzamientos de Modelos Grandes de Lenguaje (LLM por sus siglas en ingles), da paso a nuevas implementaciones de Inteligencias Artificiales Generalistas (AGI). Mas allá the optimizar modelos de machine learning, el desarrollo de AGI's requiere de aplicaciones cognitivas que habiliten a las LLM a operar de forma efectiva en aplicaciones del mundo real.

Esta memoria intrduce la Máquina de Congición-Ejecución(MCE) como un marco teórico que descompone el diseño de un AGI como un problema the aproximacion definido por 3 variables clave: El *espacio de ejecución*, textitespacio de cognición y *espacio de ejecución*. Como aproximación a este marco teórico se propone *[SystemName]* como un entorno de desarrollo para probar y diseñar AGI's implementando diferentes aproximaciones en multiples capas. Finalmente se implementan multiples técnicas incluyendo *Prompt Engineering*, codigo *Exelent*, o *Auto-Entrenamiento* con el fin de construir un AGI que soporte la interacción usuario-IA en un entorno no controlado.

#TODO: Incluir en el abstract mencion a la implementacion en RaspBerry, y resultados

ACRONYMS

- **AI**: Artificial Intelligence
- **AP**: Agent Protocol
- A_1 : Execution Layer Algorithm
- A_2 : Cognition Layer Algorithm
- **LLM**: Large Language Model
- **ECM**: Execution-Cognition Machine
- **AGI**: Artificial General Intelligence
- **RAG**: Retrieval Augmented Generation
- **PMMA**: Profile-Memory-Plan-Action

CONTENTS

1	Introducción	1
1.1	Inteligencia Artificial	1
1.2	Fundamentos de la IA Moderna	5
2	Estado del Arte	8
2.1	IA Suave y Fuerte	8
2.2	Agentes y Cognición	8
2.3	GPS vs AGI	10
2.4	Integración de Modelos de Lenguaje en Arquitecturas Cognitivas .	11
2.5	Componentes de Arquitecturas Cognitivas	13
2.6	Mecanismos Canónicos en Arquitecturas Cognitivas	14
3	Objetivos	17
3.1	Descripción del Problema	17
3.2	Requisitos	18
3.3	Metodología	19
3.4	Plan de Trabajo	20
4	Plataforma de Desarrollo	22
4.1	Arch Linux	22
4.2	Kit de Desarrollo	22
4.3	Python	24
4.4	Langchain/LangSmith	24
4.5	PyAutoGUI/Pynput	25
4.6	OpenCV / Pillow	26
5	Fundamento Teórico	27
5.1	Implementación de una MCE en un sistema cognitivo	30
5.2	Técnica de Expansión $\mathbb{B}^*$	32
5.3	Máquina de Cognición-Ejecución	35
6	Arquitectura Software	38
6.1	Capas y Módulos de la MCE	38
6.2	Espacio de Acciones	40
6.2.1	Acciones Elementales	41
6.2.2	Espacio de Acciones $\mathbb{A}^*$	43

6.3	Capa de Ejecución	46
6.3.1	Task-Sequence Protocol (TSP)	46
6.3.2	Exelent	47
6.3.3	Intérprete	49
6.4	Capa de Cognición	50
6.4.1	Autodescriptor de Acciones	51
6.4.2	Buffer de Memoria	53
6.4.3	Estado Cognitivo	54
6.4.4	Fast Agent-Protocol (FastAP)	55
6.5	Capa de Núcleo	57
6.5.1	Registro de Entidades	57
6.5.2	Servicio Maestro-Esclavo	59
6.5.3	Ejemplo de Funcionamiento de la MCE	59
6.6	Otros Componentes	61
6.6.1	Instalador Modular Automático	61
6.6.2	Imagen y Kernel para Sistemas Embebidos	63
6.6.3	Integración Continua	65
7	Experimentación	66
7.1	Aproximaciones Agénticas	66
7.2	Características Técnicas y Funcionales	69
7.3	Hipótesis del Experimento	71
7.4	Entorno de Pruebas y Evaluación	72
	Bibliography	75

1. INTRODUCCIÓN

La palabra "*inteligencia artificial*" ha sido seleccionada por el diccionario Collins como palabra del año en el 2023. Miles de empresas ahora están integrando tecnologías con "IA". Del mismo modo, múltiples medios de comunicación informan de los posibles problemas y riesgos de estas tecnologías en caso de no ser implementadas de forma apropiada. A pesar de su creciente popularidad en los últimos años, el concepto de inteligencia artificial es difícil de definir y categorizar incluso dentro del sector científico.

Dado este contexto de creciente interés y debate, el presente capítulo abordará los fundamentos de la inteligencia artificial, definiendo sus características principales, su surgimiento y las interpretaciones formales más extendidas. Este marco conceptual es clave para que el lector pueda comprender el estado del arte, que será tratado en profundidad en los capítulos siguientes.

1.1 Inteligencia Artificial

Las dos palabras acuñadas en el concepto de Inteligencia Artificial (tanto, "Inteligencia" como "Artificial") hacen referencia a dos campos de estudio totalmente diferentes. Esta composición lingüística se mantiene constante en múltiples otros idiomas (véase la terminología en inglés: "*Artificial Intelligence*"). Entender el significado de ambos conceptos es crucial para poder identificar el potencial de las tecnologías de IA y como esta se distingue respecto a tecnologías anteriores.

En este sentido, la *inteligencia* ha sido estudiada históricamente por ramas como la psicología, filosofía o educación donde se converge en múltiples definiciones que aunque distintas, resultan "intuitivamente" fáciles de relacionar: la inteligencia hará referencia a la capacidad para entender, comprender o resolver problemas, al conjunto de habilidades cognitivas que incluyen la autoconciencia, creatividad o razonamiento lógico. Fuera del ámbito científico es posible distinguir "de forma intuitiva" aquellas entidades que demuestran inteligencia de aquellas que no. Como ejemplo, aunque puede conformarse un debate en torno a las capaci-

1.1. Inteligencia Artificial

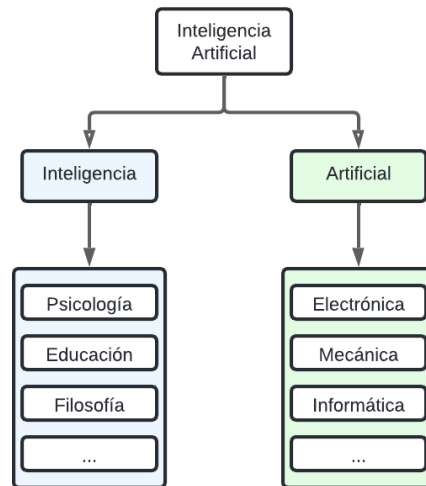


Figure 1.1: "Inteligente" vs "Artificial". Elaboración Propia

dades intelectuales de dos especies de similar origen (como lo podría ser un delfín y un tiburón); es posible afirmar con certeza que una célula vegetal es menos inteligente que un humano.

De forma similar, todo aquello que es *artificial* hace referencia a aquello que no es natural, generalmente en relación con artefactos o productos creados con un propósito determinado. Mientras que un delfín se puede considerar producto de la naturaleza, un robot aspirador se considera artificial al ver que este es consecuencia de una composición de elementos electrónicos diseñados y organizados por el ser humano con un fin concreto: limpiar el escenario en el que se encuentre.

Así pues, cuando hablamos de inteligencia artificial, haremos referencia a una simbiosis de ambos conceptos, esto es, un sistema con capacidades cognitivas como el razonamiento en la resolución de problemas compuesto por la sincronización de elementos sincronizados y diseñados por el ser humano. Esta definición de inteligencia artificial empezó a tomar forma en los años 50 tras la Conferencia de Dartmouth organizada en 1956 donde, el matemático John McCarthy junto con otros investigadores como Marvin Minsky, definieron por primera vez las bases de la inteligencia artificial:

"El estudio procederá sobre la base de la conjetura de que cada aspecto del aprendizaje o cualquier otra característica de la inteligencia puede, en principio, ser descrito con tal precisión que se pueda crear una máquina capaz de simularlo." McCarthy et al. (2006)

1.1. Inteligencia Artificial

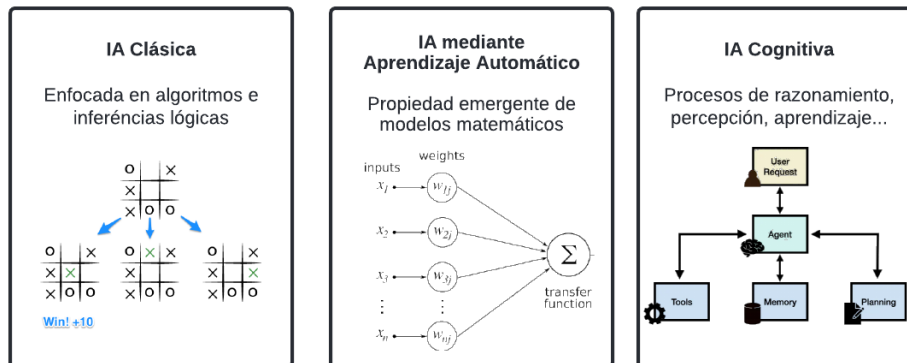


Figure 1.2: Interpretaciones de Inteligencia Artificial. Elaboración Propia

A lo largo de las décadas posteriores, el concepto de inteligencia artificial (IA) podrá definirse entorno a 3 interpretaciones de distintos sectores:

- Desde la perspectiva de la IA clásica, desarrollada principalmente por los campos de las matemáticas e informática, la inteligencia artificial se concibe como un conjunto de algoritmos predefinidos, reglas de inferencia y razonamientos lógicos que buscan tomar decisiones, realizar tareas complejas o resolver problemas de manera autónoma. El objetivo de la IA clásica es resolver tareas como encontrar el camino más corto entre dos puntos o seleccionar el movimiento con mayor probabilidad de éxito en un juego como el ajedrez.
- Desde la perspectiva del aprendizaje automático, la inteligencia artificial se define como la "propiedad de alto nivel de modelos matemáticos complejos, cuya interpretación no se basa en una secuencia de pasos predeterminada sino en las interacciones del modelo como conjunto". Esta interpretación se puede ejemplificar con una bandada de pájaros, donde las "formas" que se generan entre tales no son una propiedad de ninguno de los pájaros por separado, sino una propiedad que emerge del conjunto como una totalidad. De forma equivalente, para el aprendizaje automático la IA emerge como resultado de la totalidad de interacciones entre diferentes entidades matemáticas.¹ De aquí en adelante, haremos referencia a este concepto de capacidades abstractas del modelo como *propiedades emergentes* del modelo.

¹Cabe señalar que, bajo este enfoque, la IA no puede ser controlada de forma determinista, ya que los modelos matemáticos no definen en su totalidad el comportamiento resultante de su aplicación.

1.1. Inteligencia Artificial

- Desde la perspectiva del campo de la cognición (interpretación que se utilizará de aquí en adelante), la inteligencia artificial es un sistema que replica habilidades similares a las funciones realizadas por el cerebro humano. Dado este enfoque, el funcionamiento interno de la inteligencia no está explícitamente definido, pues se considera que la "inteligencia" reside en el proceso de resolución de problemas y no en los datos o mecanismos que contiene. Esto implica, la inteligencia se manifiesta en la metodología con la que dicho sistema "piensa", en los procesos intermedios de razonamiento, aprendizaje y percepción independientemente de los procesos internos utilizados.

El desarrollo de la inteligencia artificial desde sus distintas interpretaciones, ha dado paso a la investigación de múltiples áreas sobre lo que se consideraba "inteligente" y como estos sectores podían ser automatizados, esto es, convertirlos en elementos artificiales. Entre las áreas de estudio destacadas encontramos investigaciones sobre la representación del conocimiento, la inferencia, o la resolución de problemas.

De entre todos los objetivos de la inteligencia artificial, la resolución automática de problemas destaca aun a día de hoy como una de las propiedades más difíciles de alcanzar. Esto se debe a que resolver un problema, a diferencia de otras tareas, implica resolver múltiples sub tareas que son fáciles de para un humano pero a su vez difícilmente automatizables. Como ejemplo, la tarea de distinguir los comentarios negativos de los positivos en un foro de internet, resulta trivial de forma intuitiva, sin embargo matices sutiles del lenguaje como la ironía, gracia, emoción o exageración son problemas de gran complejidad para su resolución por un sistema ejecuta un algoritmo determinista.

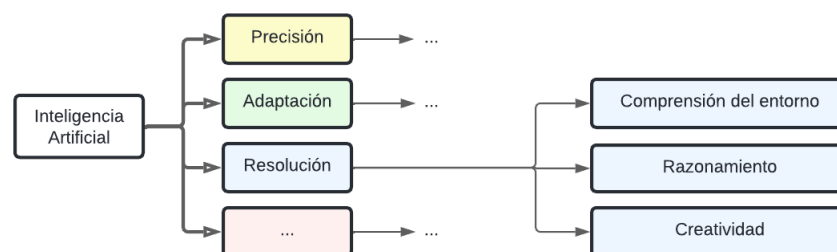


Figure 1.3: Requisitos de la IA. Elaboración Propia

1.2 Fundamentos de la IA Moderna

Entre otras arquitecturas de inteligencia artificial, los Modelos Grandes de Lenguaje (LLM por sus siglas en inglés), han destacado en los últimos años por su capacidad de adaptarse a múltiples aplicaciones. Utilizando el enfoque de inteligencia artificial basada en aprendizaje automático, estos modelos son capaces de generar imágenes, entender matices del lenguaje como la ironía, razonar matemáticamente o incluso generar modelos 3D a partir de descripciones de un objeto.

Independientemente de su estructura interna, podemos razonar que estos sistemas actúan de forma inteligente (acorde al razonamiento establecido en la sección 1.1). Por esta razón, muchas empresas han adoptado tecnologías de IA que les permiten ahorrar tiempo, esfuerzo y, en algunos casos, mejorar la calidad de sus tareas. Entre estas tareas automatizadas se encuentra la generación automática de texto, donde la IA recibe un *prompt*, esto es, una entrada en formato de texto que contiene las instrucciones que debe seguir, y responde de forma automática. Esta respuesta puede utilizarse, por ejemplo, para actualizar contenido en redes sociales o resumir textos largos.

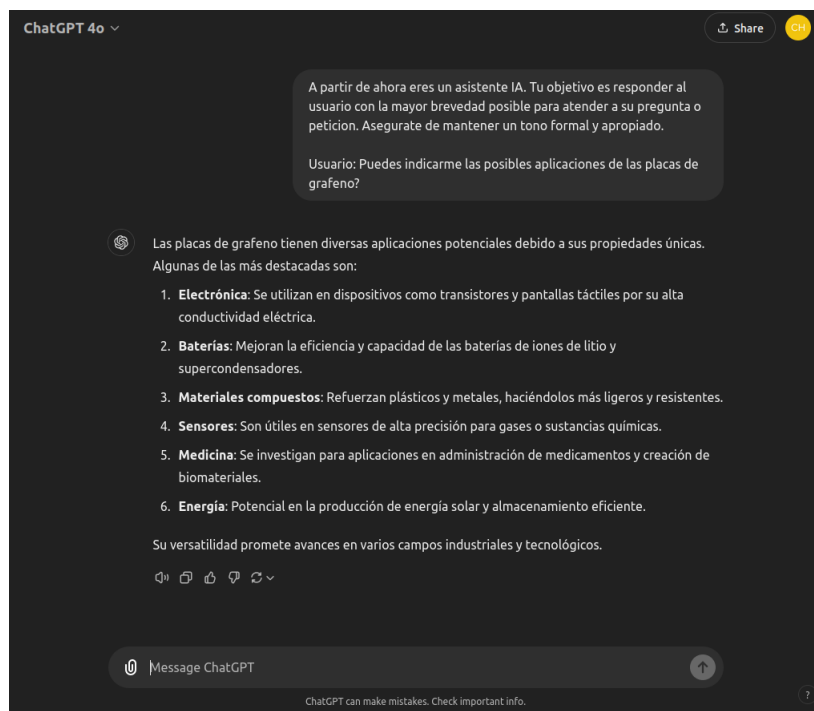


Figure 1.4: Interacción automatizada con usuario utilizando ChatGPT (modelo GPT-4o-mini). Elaboración Propia

1.2. Fundamentos de la IA Moderna

Detrás de los modelos de lenguaje actuales subyacen arquitecturas avanzadas basadas en redes neuronales profundas. A diferencia de otras tecnologías, estas arquitecturas requieren un proceso intensivo de entrenamiento, optimización y refinamiento con el fin de garantizar tanto su robustez como su eficiencia. En este contexto, diferentes compañías proporcionan servicios basados en cómo han refinado y optimizado dichas arquitecturas.

Un ejemplo es OpenAI, que ofrece acceso a sus modelos razonadores a través de su página web. Los modelos más recientes de esta serie destacan por su capacidad superior para la comprensión del lenguaje natural, razonamiento y control en tareas complejas. En contraste, la empresa Anthropic, ofrece proporciona acceso a sus modelos Claude Haiku, diseñados para ofrecer respuestas rápidas y con un enfoque en la optimización de costos, según lo informado en su plataforma en línea. Cada compañía, por tanto, implementa variaciones en el diseño y entrenamiento de sus modelos, lo que conduce a distintas capacidades y enfoques en el mercado de modelos de lenguaje.

Para aprovechar en su totalidad el potencial de los modelos de lenguaje (LLMs), surge el concepto de Agente. Un agente es una tecnología diseñada para conectar a las LLMs con diversos campos de aplicación. En esencia, un agente contiene una inteligencia artificial capaz de razonar y tomar decisiones basadas en las posibles acciones a implementar. Este posteriormente, utilizará herramientas pre-programadas en su arquitectura para traducir esos razonamientos en aplicaciones concretas. Por ejemplo, un agente puede utilizar herramientas con las que acceder a bases de datos para ofrecer respuestas a preguntas específicas de una empresa.

Para que estos agentes tengan un rendimiento óptimo, estos pasan por diversas fases de razonamiento comúnmente denominadas arquitecturas agénticas, que les permiten interactuar tanto con usuarios como con otros objetos o incluso agentes. A lo largo de este proceso, el agente evalúa la situación y ejecuta las acciones más adecuadas según las herramientas disponibles en su estructura. Un ejemplo de esta tecnología se encuentra en GEMINI ADVANCED, un agente desarrollado por Google. Este agente es capaz de procesar las solicitudes del usuario a través de comandos de voz en un dispositivo móvil, y ejecuta las acciones pertinentes para cumplir con la petición. La arquitectura de este agente le permite interpretar el lenguaje natural del usuario y, utilizando las herramientas integradas en su sistema, realizar las tareas necesarias de forma eficiente.

En la presente memoria se propone implementar una Máquina de Cognición y Ejecución (MCE) en arquitecturas de bajo coste, esto significa, una arquitectura agéntica que permita a una inteligencia artificial interactuar con su entorno (orde-

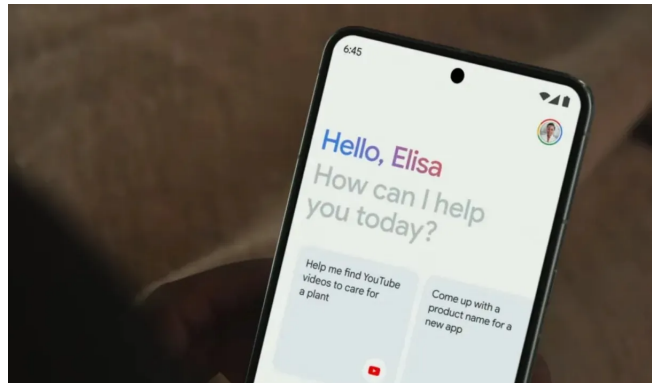


Figure 1.5: Interacción móvil con Gemini Advance. Fuente: es.digitaltrends.com/.

nadores, móviles, etc) de forma generalista y sin requerir de recursos excesivos.

Este estudio se realizará desde un enfoque basado en la interpretación cognitiva de la inteligencia artificial, implicando que se realizara un análisis basado en procesos de razonamiento y toma de decisiones, en lugar de centrarse en la estructura interna de los modelos de lenguaje.

En el siguiente capítulo, se abordarán los principales conceptos y tecnologías relevantes para el diseño de arquitecturas agénticas. Se realizará una revisión de las tecnologías actuales y sus capacidades cognitivas, analizando cómo se diferencian los agentes de propósito general de la inteligencia general artificial. Además, se explorarán las técnicas de comportamiento que permiten a los modelos de lenguaje tomar decisiones de forma autónoma y eficiente. Finalmente, se examinará el estado actual de las arquitecturas cognitivas, enfocándose en sus limitaciones y los avances que han permitido una mayor capacidad de razonamiento y adaptabilidad en agentes de IA.

2. ESTADO DEL ARTE

2.1 IA Suave y Fuerte

Para definir la Inteligencia Artificial Searle (1980) distingue dos tipos de IA. La IA *suave*¹ hace referencia a un conjunto de algoritmos o herramientas que aproximan una solución a un problema determinado utilizando estrategias basadas en un análisis previo a través de múltiples estados. En contraste, la IA *fuerte* está caracterizada por sus capacidades para entender de forma profunda, razonar y reaccionar a estados impredecibles, adaptándose y aprendiendo a lo largo del proceso de resolución del problema.

Desde el diseño de las *redes neuronales* en 1949, nuevas técnicas para el desarrollo de IA fuerte han estado en auge. Aunque la IA fuerte muestra propiedades aparentemente utópicas, muchos de los subproblemas que encierra su diseño han sido abordados por sectores originalmente distintos a la IA. Como ejemplo de esto, el campo del aprendizaje automático se ha expandido creando nuevos campos de estudio como el aprendizaje profundo. Será este nuevo campo el responsable de la creación de los Modelos Grandes de Lenguaje (LLMs), donde modelos como GPT-4 (Achiam et al. (2023)), LLAMA2 o (Touvron et al. (2023)), o GEMINI (Saeidnia (2023)) utilizan el entrenamiento de millones de iteraciones sobre el conocimiento humano para recrear propiedades emergentes como lo son el entendimiento del sentido común, habilidades de razonamiento y resolución de problemas, generación de respuestas con contexto, etc. Estos modelos aún están en etapa de desarrollo, donde el objetivo apunta a propiedades como la multimodalidad², reconocimiento de patrones y/o explicabilidad.

¹Nótese que aunque similares, es importante no confundir IA suave con IA clásica.

²(Capacidad para trabajar con múltiples tipos de entrada, como audios o imágenes.)

2.2 Agentes y Cognición

Modificando el comportamiento de los modelos de lenguaje para que utilicen un formato estandarizado, es posible conseguir que los LLMs sean capaces de usar múltiples *herramientas*. En el ámbito de la IA, las herramientas son un conjunto de funciones o fragmentos de código que permiten que el LLM interactúe con su entorno. Gracias a esto, mientras que un LLM solo puede generar una salida a partir de una consulta dada, los agentes traducen esas salidas en acciones que modifican el mundo y, opcionalmente, la propia configuración del agente.

”Los agentes son tan buenos como las herramientas que tienen.”
(LangChain, 2024).

El uso de agentes se ha extendido en los últimos años, con una implementación notable siendo la *Generación Aumentada por Recuperación (RAG)*. Esta arquitectura se sustenta sobre un agente conectado a una herramienta que recolecta información de una base de datos, proporcionando al LLM conocimiento específico de un campo o empresa y mejorando su precisión en entornos controlados.

La aparición de agentes IA abre habilita la oportunidad de realizar más investigaciones en el campo de la cognición. La *cognición* es una rama de la informática que investiga el desarrollo de sistemas capaces de replicar propiedades de la inteligencia humana como el razonamiento, la percepción, la memoria, la planificación y la toma de decisiones. El cognitivismo clásico (la primera aproximación hacia los algoritmos cognitivos) se ha estudiado desde 1956; sin embargo, la generalidad de los componentes necesarios para una arquitectura cognitiva ha planteado una limitación bloqueante en investigaciones previas.

”Tenemos círculos dentro de círculos. La dificultad central para la resolución de problemas eran los problemas de formulación limitada (PFL). Para enfrentarnos a ellos, recurrimos a la reestructuración. Para abordar esto, recurrimos a la analogía y en su núcleo encontramos la resolución de un PFL como un componente crucial. El análisis y la formalización se han visto seriamente frustrados en todo este esfuerzo.” Vervaeke (1999)

A partir de los intentos de definir alguna forma de inteligencia cognitiva, han surgido múltiples áreas de estudio que evitan depender de un sistema completamente inteligente. Por ejemplo, la resolución de problemas se ha abordado utilizando IA clásica (Russell and Norvig, 2016), STRIPS (Fikes and Nilsson, 1971),

o PDDL (Aeronautiques et al., 1998). Aunque estos enfoques han sido exitosos en múltiples aplicaciones, la llegada de agentes de IA abre la puerta a retomar el estudio original de la cognición.

Gracias a los LLMs, es posible superar las restricciones de la IA clásica y abordar el razonamiento como un problema de optimización computable. Será aquí donde los agentes de IA permitirán que los LLMs se integren en un sistema cognitivos mas complejos.

2.3 GPS vs AGI

El *General Problem Solver (GPS)* ha sido estudiado en el campo de la cognición desde 1959 con Newell et al. (1959), quien introdujo el GPS como un algoritmo hipotético o conjunto de técnicas que descomponen un problema en la ejecución de una secuencia de operadores que, combinados de la manera adecuada, pueden explorar multiples estados y subobjetivos del problema hasta alcanzar una solución válida. Aunque se lograron avances en el diseño de un GPS, las investigaciones sobre este tema se discontinuaron debido a las limitaciones de la IA clásica y la cognición descritas en la Sección 2.2.

Con propiedades similares al GPS, la *Inteligencia Artificial General (AGI)* es un nuevo enfoque basado en las técnicas de IA moderna y que pretende entender, aprender y aplicar conocimientos en cualquier tarea cognitiva, emulando las capacidades de un ser humano. Nótese que una AGI se centra en replicar la inteligencia humana en un sentido más amplio que una exploración de estados, donde la capacidad de resolución de problemas emerge como parte de la generalización del conocimiento humano, donde se requiere un nuevo marco que interactúe con esta AGI para llevar esas capacidades a la realidad.

Es importante tener en cuenta que aunque la AGI no requiere necesariamente interacción con el entorno. Sin embargo multiples interpretaciones de la AGI se pueden ver en funcion del entorno de trabajo.

"Demostrar que un sistema puede realizar un conjunto requerido de tareas en un nivel de rendimiento dado debería ser suficiente para declarar al sistema como AGI; el despliegue de dicho sistema en el mundo abierto no debería ser inherente a la definición de AGI." Morris et al. (2023)

Algunos enfoques de la AGI se centran en especializar el conocimiento general

2.3. GPS vs AGI

en una aplicación específica, donde se encapsula un agente dentro de un entorno que guía al modelo de lenguaje informándole sobre el estado actual del entorno o las herramientas disponibles.

Un ejemplo de un entorno especializado es VOYAGER. Este agente utiliza una arquitectura basada en tres módulos: el *currículum automático*, que describe el estado actual del agente y guarda información relevante; el *mecanismos de prompting iterativos*, que mantiene un bucle de retroalimentación entre las acciones codificadas por la IA y la retroalimentación obtenida; y la *biblioteca de habilidades*, que permite a la IA aprender y almacenar acciones previas y subobjetivos completados para fomentar la reutilización de herramientas. Utilizando esta arquitectura, VOYAGER demuestra la capacidad de jugar de manera autónoma al videojuego *Minecraft*, logrando múltiples objetivos solicitados.

“VOYAGER exhibe un rendimiento superior en el descubrimiento de nuevos ítems, desbloqueo del árbol tecnológico de Minecraft, recorrido de diversos terrenos y aplicación de su biblioteca de habilidades aprendidas a tareas no vistas en un mundo recién creado. VOYAGER sirve como punto de partida para desarrollar agentes generalistas potentes sin necesidad de ajustar los parámetros del modelo.” Wang et al. (2023)

Otro proyecto relevante es AUTOGPT. Este proyecto facilita el despliegue de agentes autónomos para tareas menores. Maneja la gestión de tareas, la selección de herramientas, múltiples técnicas de prompting y más. Para el despliegue de agentes, AUTOGPT proporciona lo que se conoce como la FORGE, que conecta automáticamente todos los mecanismos y servidores necesarios no solo para empezar a ejecutar el agente, sino también para proporcionar al usuario diversas herramientas para interactuar y depurar en tiempo real.

“AutoGPT utiliza el concepto de apilamiento para llamarse recursivamente a sí mismo [...], usando este método y con la ayuda de GPT 3.5 y GPT 4, crea proyectos completos iterando sobre sus propios prompts.” Fezari and Ali-Al-Dahoud (2023)

Al igual que AutoGPT, los entornos basados en AGI se están extendiendo a proyectos para la asistencia de software en empresas (MetaGPT, Wu (2023)), despliegues autónomos (SuperAGI, TransformerOptimus (2023)), asistencia en el diseño-creatividad (AgentGPT, Reworkd (2023)), y más. Con el fin de estandarizar la terminología utilizada, de aquí en adelante haremos referencia a estos entornos basados en AGI como *arquitecturas cognitivas*.

2.4 Integración de Modelos de Lenguaje en Arquitecturas Cognitivas

En secciones anteriores se han explorado distintos entornos que aprovechan el potencial de arquitecturas cognitivas. Sin embargo, para poder "configurar" estas arquitecturas, se han plantado diversos métodos para modificar o *tunear* el comportamiento de un modelo de lenguaje con el fin de que estos puedan implementarse con éxito. Así pues, a continuación se destacan las técnicas clave en el estado del arte para construir agentes a partir de LLMs:

- **Tuneado (Fine-Tuning):** Todos los LLMs consisten en una o varias capas en una red neuronal basada en aprendizaje profundo. Estas capas contienen un conjunto de parámetros que definen el conocimiento o el comportamiento del modelo. Al entrenar un LLM ya estable con un conjunto de datos específicos de un campo, podemos mejorar la precisión del modelo con el conocimiento proporcionado y definir el formato y/o las directrices que debe seguir. Aunque este método obtiene mejores resultados que las siguientes técnicas, puede perder propiedades emergentes del LLM original y requiere un análisis previo del conjunto de datos proporcionado (sesgos, expectativas vs. resultados, limpieza de datos, etc.). En este campo, técnicas como el entrenamiento personalizado o el *congelamiento* de capas pueden ser usadas para mejorar los resultados del ajuste fino.
- **Ingeniería de Prompts (Prompting):** Sin modificar los parámetros del LLM, todavía podemos cambiar el comportamiento esperado introduciendo un *prompt* diseñado para un objetivo específico que la IA recibirá como entrada y utilizará como directrices. Aunque este método no garantiza una mejor precisión en comparación con el uso de fine-tuning, no altera los parámetros del modelo y permite la definición de estructuras de razonamiento más complejas. Es importante señalar que, aunque este método puede generar riesgos de seguridad debido a un mal comportamiento de la IA, técnicas como la Cadena de Pensamiento (Chain of Thought, CoT, ReAct), la Generación Aumentada por Recuperación (RAG) o los *Few-Shot prompting* conducen a propiedades de razonamiento de alto nivel que no se han logrado con otros métodos. Sahoo et al. (2024) profundiza más en este campo.
- **Excitación del Tiempo de Computo (Test-Time Compute):** Mediante mecanismos combinados de prompting y técnicas de fine-tuning, es posi-

2.5. Componentes de Arquitecturas Cognitivas

ble ajustar el comportamiento de los agentes para que iteren de forma exhaustiva sobre pasos de razonamiento y herramientas disponibles. Este tiempo de generación de razonamiento se conoce como *tiempo de computo*, y se refiere al tiempo que la IA necesita para generar una respuesta. Al excitar el tiempo latente, se puede aumentar la precisión de la IA al permitirle explorar más estados y subobjetivos antes de generar una respuesta Norris Jr et al. (1978).

- **Mezclado de Expertos:** Al utilizar tanto la ingeniería de prompts como los métodos de fine-tuning, es posible optimizar múltiples agentes dividiendo el comportamiento esperado en varios subobjetivos. A estos agentes se les suele llamar *Expertos*, y reducen las alucinaciones de los LLMs distribuyendo la atención necesaria para completar una consulta dada en múltiples ejecuciones en lugar de una sola instancia. Algunos marcos de AGI, como AutoGen Wu et al. (2023), están completamente basados en este método.
- **Excitación de Características Interpretables:** Al usar LLMs, las características abstractas parecen tener una relación con patrones visibles dentro de los parámetros de la red neuronal. Estos parámetros pueden ser excitados (por ejemplo, aumentando la influencia de esos parámetros en la salida) para regular comportamientos relacionados con esa característica. La investigación realizada por Viteri et al. (2024) en este tema abre la puerta a usar este método en futuros diseños de agentes.

2.5 Componentes de Arquitecturas Cognitivas

Usando IA clásica y aprendizaje por refuerzo, Laird (2019) introdujeron SOAR como una arquitectura cognitiva unificada que integra varias funciones cognitivas, como el aprendizaje, la memoria y la resolución de problemas, en un único marco. SOAR emplea un ciclo de decisión que implica proponer, evaluar y seleccionar operadores en función del estado actual.

En la investigación contemporánea, las arquitecturas cognitivas han evolucionado al incorporar más herramientas e integrar las capacidades de los LLMs como mecanismos principales. Wang et al. (2024) proporcionan una encuesta exhaustiva de las principales arquitecturas para despliegues de AGI en los últimos años, describiendo el modelo PMPA como un marco unificado que abarca todas las arquitecturas estudiadas.

2.6. Mecanismos Canónicos en Arquitecturas Cognitivas

El modelo PMPA aborda cuatro temas principales que deben ser implementados por la arquitectura cognitiva:

- *Perfil*: Define las principales directrices y reglas para los agentes implementados, orientados a los objetivos principales, la base de conocimientos y el comportamiento.
- *Memoria*: Define la información y los datos obtenidos del entorno del agente y establece una estructura y mecanismo para recuperar, codificar y clasificar el conocimiento relevante.
- *Planificación*: Define el mecanismo que permite al agente descomponer el objetivo principal en múltiples subobjetivos, emulando las capacidades de planificación humana.
- *Acción*: Define el mecanismo que conecta o traduce las órdenes solicitadas por el agente en un conjunto de herramientas que interactuarán con el entorno o comportamiento del agente.

Es importante señalar que, a diferencia de SOAR, este marco no prescribe métodos específicos para conectar cada módulo. En su lugar, describe las propiedades primarias que una arquitectura cognitiva debe poseer para ser viable en un despliegue de AGI. Además del modelo PMPA, existen otros marcos de AGI, como el Protocolo de Agentes-AI-Engineer-Foundation (2023)-, que proponen estándares alternativos de API, comportamiento y módulos. Sin embargo, se necesitan más avances en las arquitecturas cognitivas para establecer qué estándar será finalmente adoptado.

2.6 Mecanismos Canónicos en Arquitecturas Cognitivas

En los últimos años han emergido una multiples de mecanismos funcionales que, si bien no constituyen aún un estándar formal, están siendo ampliamente adoptados en arquitecturas cognitivas. Estos mecanismos permiten una integración más eficaz de los LLMs dentro de sistemas complejos, facilitando la interacción, el razonamiento y la gestión temporal del contexto.

A continuación, se describen algunos de los mecanismos que actualmente están tomando un rol canónico en estos entornos.

2.6. Mecanismos Canónicos en Arquitecturas Cognitivas

- **Messaging:** Para interactuar de manera estructurada con modelos de lenguaje se ha adoptado ampliamente un sistema de mensajería etiquetada que asigna roles explícitos a cada uno de los participantes en la conversación. Esta estrategia se basa en definir un perfil para cada fragmento del texto, permitiendo al modelo distinguir entre distintos tipos de información y, con ello, mejorar la coherencia, el control y el determinismo del sistema cognitivo. Comúnmente, se utilizan tres roles principales: usuario, IA y sistema. El rol de usuario representa a los participantes humanos o a la información proveniente del entorno; el rol de IA agrupa las respuestas generadas por el modelo; y el rol de sistema contiene instrucciones, configuraciones o información de contexto proporcionadas por el desarrollador para guiar el comportamiento del modelo. Este enfoque de mensajería ha sido ampliamente estandarizado en arquitecturas agénticas modernas, como LangChain u Ol-lama, facilitando la implementación de interacciones más robustas y modulares entre componentes.
- **Tool o Function Calling:** El mecanismo de Tool Calling o Function Calling permite a los modelos de lenguaje delegar tareas específicas a funciones externas, actuando como intermediarios que interpretan, generan o deciden cuándo invocar herramientas programáticas. Es gracias a este mecanismo que se logra la transición desde un modelo puramente basado en texto hacia un agente capaz de interactuar con su entorno. Su funcionamiento se basa en que el modelo genera una estructura de llamada a función (habitualmente en formato JSON) que incluye la información como el nombre de la herramienta y los parámetros requeridos, la cual puede ser interpretada por el entorno de ejecución para activar funciones concretas. Así pues, este enfoque amplía significativamente las capacidades del modelo al integrarlo con herramientas que pueden ejecutar acciones, recuperar datos o interactuar con otros sistemas.
- **Prompt Estructurado:** El prompt estructurado (o structured prompting) es una técnica que permite guiar a los modelos de lenguaje para que generen salidas en formatos predefinidos, usualmente estructurados según esquemas u otros marcos de representación. A través de esta estrategia, se establecen plantillas o instrucciones explícitas que inducen al modelo a producir respuestas conformes a una estructura semántica esperada, lo que facilita su posterior interpretación, validación y uso programático. Este mecanismo resulta fundamental en entornos donde el modelo actúa como componente intermedio dentro de una arquitectura mayor, ya que permite traducir el lenguaje natural en representaciones formalizadas que pueden ser procesadas por mecanismos externos.

2.6. Mecanismos Canónicos en Arquitecturas Cognitivas

- **Guardrails:** Los Guardrails constituyen un conjunto de mecanismos diseñados para restringir, supervisar y orientar el comportamiento de los modelos de lenguaje durante la generación de texto. Su objetivo principal es garantizar que las respuestas del modelo se mantengan dentro de límites seguros, coherentes y alineados con los objetivos del sistema o las políticas establecidas por el desarrollador. Estos mecanismos pueden adoptar múltiples formas, incluyendo validaciones de salida, filtros semánticos, restricciones en el flujo conversacional o la verificación de estructuras y formatos. En el contexto de arquitecturas agénticas, los Guardrails son fundamentales para preservar la integridad del sistema, reducir comportamientos inesperados o erráticos, y asegurar el cumplimiento de normas éticas, técnicas o de dominio específico.

3. OBJETIVOS

3.1 Descripción del Problema

El presente proyecto plantea la construcción teórica y práctica de una Máquina de Cognición-Ejecución (MCE): Una arquitectura cognitiva diseñada para abordar tareas propuestas por el usuario, planificar estrategias de resolución, adaptarse a condiciones cambiantes del entorno y llevar a cabo acciones computacionales que conduzcan a una solución satisfactoria.

A diferencia de los enfoques tradicionales centrados exclusivamente en el procesamiento simbólico o en la generación de texto, la MCE se concibe como un sistema integrado que combina razonamiento, percepción y ejecución. Su propósito no es únicamente resolver tareas, sino hacerlo de forma autónoma, interactuando continuamente con un entorno operativo. Esta interacción requiere no solo la interpretación precisa de las instrucciones y la disponibilidad de acciones programables, sino también la capacidad de adaptación reactiva frente a imprevistos, lo cual impone exigencias particulares sobre el diseño de la arquitectura cognitiva subyacente.

El problema central que aborda este proyecto es, por tanto, el diseño y validación de una arquitectura cognitiva generalista que pueda ser implementada en un sistema físico, el cual funcionará como interfaz entre el usuario, el entorno y los módulos de decisión de la MCE. Para ello, se propone una aproximación progresiva que comienza con el análisis teórico de los principios fundamentales de la arquitectura, continúa con el desarrollo de un entorno experimental para evaluar distintas configuraciones e implementaciones, y culmina con la integración de un prototipo funcional y parcialmente completo, entendiéndose por completo su integración desde un nivel hardware con bajos recursos a una implementación software que satisfaga las propiedades de la MCE.

Este prototipo, aunque limitado en términos de accesibilidad general, permitirá demostrar la viabilidad técnica y escalabilidad del concepto, con vistas a su futura aplicación en dispositivos de uso cotidiano como ordenadores personales o teléfonos inteligentes.

Para abordar el problema descrito, este proyecto se plantean los siguientes objetivos técnicos:

- Definir de forma rigurosa y completa el fundamento teórico que sustenta el modelo de MCE propuesto.
- Desarrollar un entorno modular y experimental que permita probar distintas arquitecturas cognitivas para la implementación de la MCE.
- Integrar, a nivel software, las capas de Cognición y Ejecución como componentes diferenciados pero interoperables de la arquitectura.
- Implementar la MCE en un microprocesador de bajo coste, concretamente un ARM Cortex-A55, validando su funcionamiento en hardware embebido.
- Desplegar la MCE en un entorno operativo no controlado, como un sistema de escritorio, en el que pueda ejecutar acciones tales como abrir aplicaciones o navegar por internet.
- Habilitar la interacción entre el usuario y la MCE mediante una interfaz funcional, sin requerir necesariamente capacidades avanzadas de interacción humano-robot (HRI).

3.2 Requisitos

Con el objetivo de alcanzar los objetivos propuestos, se han establecido los siguientes requisitos:

- El sistema operativo principal para el microprocesador será Arch Linux, por ser un sistema de bajo consumo y con compatibilidad con entornos embebidos.
- Toda la arquitectura de la MCE deberá ser portátil y replicable en otros entornos compatibles con ARM Cortex-A55 sin necesidad de modificar el código fuente.
- La MCE deberá ser completamente operativa desde una terminal, sin requerir interfaces gráficas de usuario.

- El sistema deberá funcionar con conexión a Internet y estar capacitado para interactuar con servicios externos mediante APIs, incluyendo modelos LLM alojados en la nube.
- El modelo no asumirá disponibilidad de GPU local; los modelos de lenguaje serán accedidos vía endpoints remotos, con la computación pesada delegada a servicios en la nube.
- La MCE deberá ser capaz de abrir programas instalados, reorganizar archivos, navegar por Internet y generar programas simples a partir de instrucciones en lenguaje natural.
- La arquitectura deberá estar construida sobre una infraestructura modular basada en LangChain, permitiendo la integración con múltiples herramientas y flujos de control.
- El sistema deberá tener capacidad de ejecutar instrucciones en tiempo real, minimizando la latencia perceptible entre la entrada del usuario y la acción del sistema.
- La arquitectura cognitiva deberá estar diseñada de forma que todas las capas (cognición, ejecución, percepción, planificación) sean sustituibles e intercambiables, facilitando la investigación y experimentación modular.
- El diseño del prototipo no contemplará componentes físicos como pantallas, LEDs o botones; la interacción será exclusivamente por software y vía línea de comandos.

3.3 Metodología

El desarrollo de este proyecto se ha estructurado en torno a una metodología de trabajo que combina una fase inicial de investigación teórica con un enfoque iterativo de implementación y validación experimental, siguiendo prácticas propias de metodologías ágiles. Esta estrategia permite avanzar progresivamente hacia una versión funcional de la MCE, evaluando en cada iteración tanto la coherencia teórica como el rendimiento práctico del sistema.

En primer lugar, se ha realizado una revisión del estado del arte centrada en arquitecturas cognitivas y en la integración de modelos de lenguaje dentro de entornos

operativos. Esta revisión ha servido como base para formular una primera aproximación teórica de la MCE, que incluye la definición de sus capas funcionales y de los principios fundamentales que rigen su comportamiento.

Posteriormente, se ha desarrollado un primer esquema provisional de la arquitectura de la MCE, el cual ha guiado la implementación inicial de sus distintos módulos (Capa de Cognición, Espacio de Acciones, etc). A partir de esta versión base, el proyecto ha seguido una filosofía iterativa centrada en sprints de desarrollo, en los cuales se incorporan nuevas capacidades, se integran arquitecturas cognitivas experimentales, y se realizan múltiples pruebas para mejorar el desempeño y adaptabilidad del sistema.

A lo largo de este proceso, se han llevado a cabo numerosos experimentos sobre entornos no controlados, utilizando ordenadores convencionales como plataforma de prueba. Este enfoque busca replicar situaciones reales donde la MCE pueda enfrentarse a condiciones impredecibles y tareas diversas.

Todo el desarrollo del proyecto se gestiona mediante un repositorio de GitHub, que incluye tanto el código fuente como los distintos recursos empleados. Además, se ha construido una wiki detallada donde se documentan los avances teóricos y prácticos, así como manuales de uso de los componentes desarrollados, instrucciones de instalación y los resultados obtenidos en cada iteración experimental. Esta documentación permite un seguimiento exhaustivo del proceso y garantiza la trazabilidad de las decisiones tomadas a lo largo del proyecto.

3.4 Plan de Trabajo

Siguiendo la metodología descrita en la anterior sección, el proyecto se estructura en seis fases principales:

1. **Revisión teórica:** Análisis del estado del arte en arquitecturas cognitivas y LLMs.
2. **Diseño inicial:** Definición de la estructura modular de la MCE.
3. **Implementación base:** Desarrollo de las capas principales en entorno local.
4. **Iteración experimental:** Integración progresiva de capacidades cognitivas mediante ciclos de mejora continua.

3.4. Plan de Trabajo

5. **Despliegue en hardware:** Adaptación de la MCE a un microprocesador ARM Cortex-A55.
6. **Ajustes finales:** Validación funcional y preparación del sistema para su utilización.

4. PLATAFORMA DE DESARROLLO

En este capítulo se desarrollarán algunas de las herramientas y recursos utilizados mas relevantes a lo largo del proyecto

4.1 Arch Linux

El sistema operativo elegido para la placa en la que se desplegará la MCE es Arch Linux. La elección se fundamenta en su bajo coste computacional (lightweight), su amplia compatibilidad con herramientas modernas de desarrollo, y su flexibilidad para configuraciones personalizadas, lo cual resulta especialmente valioso durante las fases experimentales del proyecto. Al tratarse de una distribución minimalista, permite un mayor control sobre los paquetes instalados y el entorno de ejecución, optimizando así los recursos del sistema.

Aunque el proceso de instalación y configuración de Arch Linux requiere conocimientos intermedios en sistemas operativos, su adopción ofrece ventajas significativas en términos de rendimiento, estabilidad y transparencia del entorno. Además, la estructura del sistema facilita la automatización y el despliegue de servicios relacionados con la MCE.

Cabe destacar que Arch Linux funcionará como sistema anfitrión (host) para la ECM, desempeñando el rol de servidor para las Capas de Ejecución/Cognición. No obstante, el espacio de acciones (esto es, la máquina controlada) es compatible con otros sistemas operativos, como Windows, Ubuntu o Raspbian, lo que garantiza la interoperabilidad y flexibilidad del sistema en entornos heterogéneos.

4.2 Kit de Desarrollo

Para la implementación y validación de la MCE se ha optado por el uso del kit de desarrollo IMX8MPEVK, el cual proporciona acceso completo al SoC i.MX 8M Plus, que incorpora un microprocesador ARM Cortex-A53. Este kit permite

4.2. Kit de Desarrollo

realizar pruebas exhaustivas sobre un entorno embebido realista, facilitando el desarrollo, despliegue y evaluación del sistema en condiciones similares a las de un producto final.

Aunque el coste del kit de desarrollo es elevado, el microprocesador utilizado se encuentra disponible en el mercado por un rango estimado de 30 a 40 euros, lo cual se ajusta a la filosofía de bajo coste del proyecto. El propósito del uso de este kit es demostrar que la MCE puede ser ejecutada en un sistema basado en un microprocesador accesible económicamente, validando así su viabilidad como prototipo para futuras aplicaciones a escala, donde la arquitectura puede ser directamente integrada en el SoC i.MX sin necesidad del kit de desarrollo.

Se ha decidido utilizar un microprocesador en lugar de un microcontrolador por dos razones fundamentales. En primer lugar, el proyecto requiere de un sistema operativo completo que permita instalar y ejecutar múltiples herramientas de desarrollo y control, muchas de las cuales no son compatibles con microcontroladores tradicionales. En segundo lugar, el sistema debe ser capaz de soportar operaciones complejas y arquitecturas cognitivas avanzadas, que exigen una capacidad de procesamiento superior y una mayor flexibilidad en la gestión de recursos.

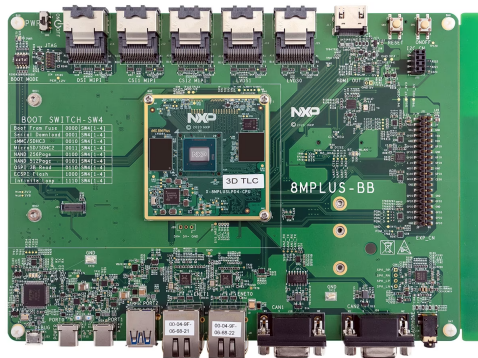


Figure 4.1: Kit de desarrollo IMX8MPEVK. Fuente: <https://www.nxp.com/>

4.3 Python

El lenguaje de programación elegido para el desarrollo de la MCE es Python, considerado actualmente el lenguaje de facto en el ámbito de la inteligencia artificial. Esta elección responde tanto a su popularidad como a su ecosistema maduro, que incluye una amplia gama de bibliotecas especializadas como LangChain, MCP y FastAPI, fundamentales para la implementación modular y conectada de arquitecturas cognitivas.

Adicionalmente, Python otorga la capacidad de diseñar sistemas complejos de forma legible y mantenible. Características como la introspección dinámica, el tipado flexible, y la posibilidad de realizar meta-programación permiten definir estructuras adaptativas, propiedad que resulta esencial para un sistema como la MCE, donde los módulos no solo deben poder reemplazarse o ampliarse sin reestructurar el sistema completo sino que además deben poder configurarse de forma automática en tiempo real.

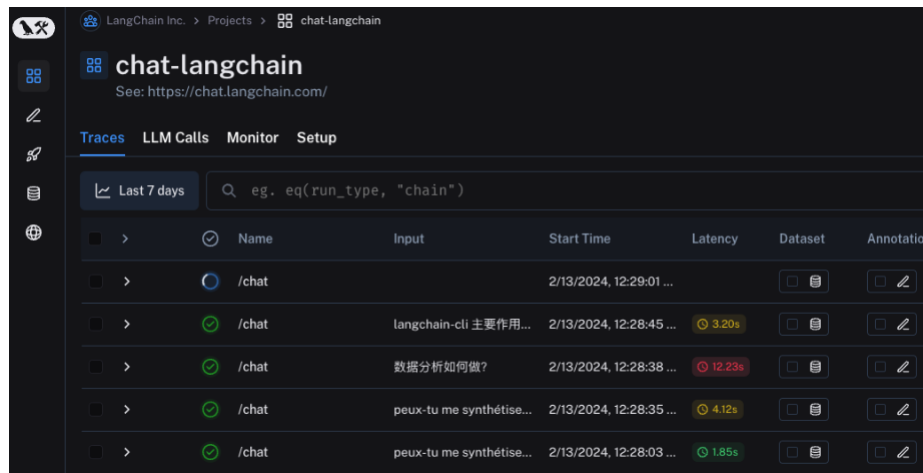
4.4 Langchain/LangSmith

Las bibliotecas LangChain y LangSmith constituyen elementos fundamentales para la implementación y monitoreo de la MCE.

LangChain actúa como capa de abstracción que permite interactuar, de forma unificada, con distintos modelos de lenguaje disponibles a través de API, tales como los ofrecidos por Anthropic, OpenAI, Ollama, entre otros. Más allá del acceso a modelos, LangChain proporciona un conjunto robusto de patrones de diseño orientados a agentes, incluyendo sistemas de mensajería, mecanismos de prompting estructurado, modelos de memoria contextual y soporte para grafos agénticos. Estas funcionalidades son esenciales para la construcción de arquitecturas cognitivas de alto nivel, facilitando la organización jerárquica y modular de los procesos cognitivos simulados por la MCE.

Complementariamente, LangSmith se integra como plataforma de observabilidad y trazabilidad del sistema. Esta herramienta permite registrar, de manera no intrusiva, el flujo completo de interacciones dentro de los agentes construidos con LangChain. A través de su interfaz gráfica, es posible visualizar de forma clara el comportamiento del sistema, incluyendo información crítica como la latencia, el coste computacional, el número de llamadas a modelos y el flujo de decisiones.

Esta capacidad resulta especialmente útil durante las fases de experimentación e iteración, ya que permite identificar cuellos de botella, evaluar el rendimiento y realizar ajustes con mayor precisión.



	Name	Input	Start Time	Latency	Dataset	Annotation
>	/chat		2/13/2024, 12:29:01 ...			
>	/chat	langchain-cli 主要作用...	2/13/2024, 12:28:45 ...	3.20s		
>	/chat	数据分析如何做?	2/13/2024, 12:28:38 ...	12.23s		
>	/chat	peux-tu me synthétise...	2/13/2024, 12:28:35 ...	4.12s		
>	/chat	peux-tu me synthétise...	2/13/2024, 12:28:03 ...	1.85s		

Figure 4.2: Interfaz de depuración de LangSmith para componentes de LangChain.
Fuente: <https://docs.smith.langchain.com/>

4.5 PyAutoGUI/Pynput

Para habilitar la interacción directa entre la MCE y el sistema operativo del cliente controlado, se emplean las bibliotecas PyAutoGUI y Pynput.

Estas bibliotecas permiten tanto la captura de información contextual como la manipulación activa del sistema, actuando como una interfaz sensorimotora entre la arquitectura cognitiva y el entorno operativo. En concreto, PyAutoGUI se utiliza principalmente en sistemas Windows y Pynput en entornos Linux, lo que asegura una cobertura multiplataforma para la ejecución del sistema.

En el plano perceptivo, estas herramientas posibilitan la captura de capturas de pantalla y el monitoreo de eventos del usuario, como movimientos del ratón o pulsaciones de teclas. Estos datos perceptuales son enviados a las capas cognitivas para su análisis e interpretación, actuando como entrada sensorial del sistema.

De forma inversa, la capa de ejecución de la MCE puede controlar activamente el entorno mediante simulación de entrada de teclado, clics de ratón o desplazamiento del cursor, lo que permite ejecutar comandos, interactuar con aplicaciones

y realizar tareas definidas por el usuario. Este mecanismo convierte a la MCE en un agente capaz de operar sobre sistemas externos de manera autónoma y efectiva, sin requerir modificaciones en el sistema operativo huésped.

4.6 OpenCV / Pillow

Dado que la MCE debe ser capaz de percibir e interpretar información gráfica del entorno se integran las bibliotecas OpenCV y Pillow como herramientas fundamentales para el procesamiento de datos visuales.

Ambas bibliotecas permiten transformar la información gráfica capturada en representaciones estructuradas que pueden ser utilizadas por la capa cognitiva del sistema. En este contexto, se han empleado para tareas como la detección y localización de iconos, la generación de regiones de interés mediante bounding boxes, y el ajuste de calidad o resolución de las imágenes, adaptando la información visual a formatos que resulten viables para su análisis automatizado.

Adicionalmente, estas herramientas también han sido utilizadas como soporte para técnicas de OCR (reconocimiento óptico de caracteres), permitiendo extraer texto de elementos gráficos que luego puede ser interpretado y utilizado por los módulos cognitivos.

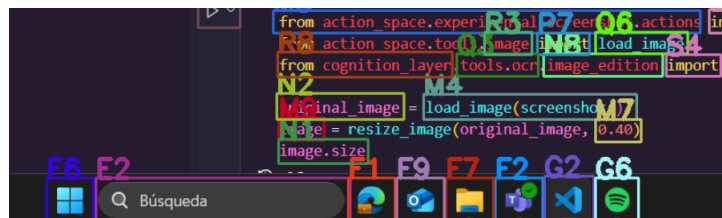


Figure 4.3: Detección de iconos detectados en pantalla usando OpenCV y Pillow.
Fuente: Elaboración Propia

5. FUNDAMENTO TEÓRICO

Una *Máquina de Cognición-Ejecución* (MCE) es un modelo computacional hipotético que opera sobre un sistema cognitivo con el objetivo de generar soluciones, compuestas por secuencias de acciones, a problemas bien definidos en un espacio formal. Este concepto se fundamenta en la idea de que la cognición puede ser modelada computacionalmente como un proceso de transformación entre representaciones abstractas de problemas y secuencias de acciones eficaces.

- La MCE no presupone una arquitectura específica, sino una relación funcional entre problemas, soluciones y acciones.
- El sistema cognitivo actúa como un mapeador entre interpretaciones de problemas y soluciones viables dentro de un conjunto estructurado.
- La hipótesis de unicidad e irreducibilidad de la representación de problemas busca eliminar ambigüedades y redundancias en el espacio de entrada.

Sea $\mathbb{P}$ un conjunto infinito e hipotético de problemas. Cada problema $p \in \mathbb{P}$ es independiente y mutuamente exclusivo respecto de los demás, es decir,

$$\forall p_i, p_j \in \mathbb{P}, p_i \neq p_j \Rightarrow \neg(p_i \equiv p_j)$$

donde $\equiv$ denota equivalencia semántica bajo una interpretación cognitiva. Asumimos que cada problema p está representado mediante una descripción única e irreducible, la cual no admite ambigüedad ni redundancia y es inconfundible con cualquier otro $p' \in \mathbb{P}$.

Sea $\mathbb{A}$ un conjunto infinito e hipotético de acciones. Cada acción $a \in \mathbb{A}$ representa una interacción elemental que un sistema cognitivo puede realizar, ya sea con su entorno o consigo mismo.

Sea $\mathbb{S}$ un conjunto hipotético de soluciones. Cada solución $s \in \mathbb{S}$ está compuesta por una secuencia finita y ordenada de acciones:

$$s = (a_1, a_2, \dots, a_n), a_i \in \mathbb{A}, n \in \mathbb{N}^+$$

Dicha secuencia es inmutable y única para la solución dada. El orden de las acciones es relevante, es decir, dos soluciones que contengan las mismas acciones en distinto orden se consideran distintas.

Para cada problema $p \in \mathbb{P}$ existe al menos un subconjunto no vacío de soluciones $\mathbb{S}_p \subset \mathbb{S}$ tal que cada $s \in \mathbb{S}_p$ satisface completamente el problema p . Notamos esta relación como:

$$s \in \mathbb{S}_p \iff s \in \mathbb{S} \wedge s \models p$$

donde $\models$ denota que s satisface (resuelve) el problema p bajo la semántica del sistema cognitivo. No se considerarán en este modelo las soluciones parciales ni grados de satisfacción.

- Esta relación establece una correspondencia semántica entre el espacio de problemas y el de soluciones.
- El modelo asume que la existencia de una solución completa implica que el problema ha sido resuelto de forma definitiva.

Cabe destacar que, aunque un problema puede tener múltiples soluciones, estas son mutuamente excluyentes, pero equivalente en tanto cada secuencia de acciones difiere estructuralmente de las demás pero todas satisfacen el mismo problema:

$$\forall s_i, s_j \in \mathbb{S}_p, s_i \neq s_j \Rightarrow s_i \equiv s_j$$

Finalmente, asumimos la existencia de al menos una solución por cada problema:

$$\forall p \in \mathbb{P}, \exists s \in \mathbb{S}_p \text{ tal que } s \models p$$

Dados los elementos previamente definidos —el espacio de problemas $\mathbb{P}$, el espacio de acciones $\mathbb{A}$ y el espacio de soluciones $\mathbb{S}$ —, es necesario introducir el

concepto de *base de conocimiento* para posibilitar su aplicación dentro de un sistema cognitivo.

Denotamos por C a una *base de conocimiento*, entendida como una abstracción teórica que no especifica su contenido ni su estructura interna, pero que puede ser empleada por sistemas cognitivos para analizar un problema $p \in \mathbb{P}$ y generar una solución $s \in \mathbb{S}_p$.

En este marco, se impone la siguiente condición sobre C :

La base de conocimiento C debe contener, representar o permitir el acceso a la información necesaria sobre el problema p que haga posible que un sistema cognitivo encuentre al menos una solución $s \in \mathbb{S}_p$ tal que $s \models p$.

Temporalmente, asumiremos que C es una *base de conocimiento ideal*, lo que implica que:

$$\forall p \in \mathbb{P}, \exists s \in \mathbb{S} \text{ tal que } s \models p \text{ y } s \text{ es derivable a partir de } C$$

Cabe destacar que C no necesariamente posee una representación simbólica explícita. En el caso de sistemas cognitivos implementados mediante redes neuronales artificiales, C puede estar distribuida de manera implícita en los parámetros internos del modelo. Así, la información relevante no reside en estructuras legibles externamente, sino como propiedades emergentes del comportamiento del sistema.

Con todos los elementos anteriores, podemos establecer una definición operativa para una *Máquina de Cognición-Ejecución* (MCE). Una MCE es un sistema cognitivo que, dado un problema $p \in \mathbb{P}$, una base de conocimiento C , y un conjunto de acciones elementales $\mathbb{A}$, es capaz de generar una solución $s \in \mathbb{S}_p$ tal que s satisface completamente el problema p .

Formalmente, se define como una función computable:

$$T : \mathbb{P} \times C \times \mathbb{A} \rightarrow \mathbb{S}$$

tal que:

5.1. Implementación de una MCE en un sistema cognitivo

$$\forall p \in \mathbb{P}, T(p, C, \mathbb{A}) = s \in \mathbb{S}_p \subseteq \mathbb{S}, \quad \text{con } s \models p$$

Es decir, la MCE implementa un proceso de transformación cognitiva que mapea un problema bien definido al conjunto de acciones resolutivas adecuadas, utilizando conocimiento previo contenido (explícita o implícitamente) en C .

5.1 Implementación de una MCE en un sistema cognitivo

Para describir cómo se implementa una MCE en un sistema cognitivo, introduciremos formalmente el concepto de *aproximador*. Sea el símbolo “*” la notación para indicar una versión aproximada o parcial de una entidad. A continuación, definiremos los dos aproximadores principales: $\mathbb{A}^*$ (aproximador de acciones) y C^* (aproximador de base de conocimiento).

Denotemos por $\mathbb{A}$ el espacio infinito e hipotético de acciones elementales. El aproximador de acciones $\mathbb{A}^*$ es un único conjunto creciente que, en cada etapa de aprendizaje o evolución del sistema, contiene un subconjunto de $\mathbb{A}$. Formalmente, podemos describirlo de la siguiente manera:

$$\mathbb{A}^* = \bigcup_{n=0}^{\infty} A_n^*, \quad A_0^* = \emptyset, \quad A_n^* \subset A_{n+1}^* \subseteq \mathbb{A} \quad ,$$

$$\text{y } \lim_{n \rightarrow \infty} A_n^* = \mathbb{A}.$$

- En la etapa inicial ($n = 0$), $\mathbb{A}^*$ contiene cero acciones: $A_0^* = \emptyset$.
- En cada etapa n , el sistema incorpora nuevas acciones $a \in \mathbb{A}$ a A_n^* , de modo que $A_n^* \subset A_{n+1}^*$.
- En el límite (cuando $n \rightarrow \infty$), el aproximador $\mathbb{A}^*$ coincide con el espacio completo de acciones $\mathbb{A}$.

Así, en cualquier instante finito, $\mathbb{A}^*$ es un subconjunto estricto de $\mathbb{A}$, pero crece a lo largo de la evolución del sistema cognitivo. Mientras $\mathbb{A}^*$ no alcance $\mathbb{A}$, habrá problemas cuya solución requiera acciones $a \notin \mathbb{A}^*$, imposibilitando resolverlos.

5.1. Implementación de una MCE en un sistema cognitivo

Sea $\mathbb{P}$ el espacio infinito e hipotético de problemas. El aproximador de base de conocimiento C^* se define en términos del *alcance* de los problemas que puede resolver. Sea $P_1 \subset \mathbb{P}$ un subconjunto de problemas; decimos que una base C_1^* tiene alcance P_1 si, para cada $p \in P_1$, existe al menos una solución $s \in \mathbb{S}$ con $s \models p$ derivable a partir de C_1^* . A medida que se “amplía” el alcance, se obtiene otro aproximador C_2^* con $P_1 \subset P_2 \subset \mathbb{P}$. Formalmente:

$$P_1 \subset P_2 \subset \dots \subset \mathbb{P}, \quad C_1^* \rightsquigarrow P_1, \quad C_2^* \rightsquigarrow P_2, \quad \text{y} \quad \bigcup_{i=1}^{\infty} P_i = \mathbb{P}.$$

- Cada C_i^* es una base de conocimiento “parcial” que resuelve el subconjunto P_i .
- Cuando $P_i \subset P_{i+1}$, la base C_{i+1}^* contiene información (explícita o implícita) para resolver más problemas que C_i^* .
- En el límite, si $\bigcup_i P_i = \mathbb{P}$, entonces la unión de todos los aproximadores equivaldría a una base de conocimiento ideal C .

En particular:

$$\forall p \in P_i, \exists s \in \mathbb{S} \text{ tal que } s \models p \quad \text{y} \quad s \text{ se deriva de } C_i^*.$$

Si un problema p no pertenece al alcance de C_i^* , entonces C_i^* no contiene la información necesaria para derivar ninguna solución s satisfactoria para p .

Al combinar los aproximadores $\mathbb{A}^*$ (acciones parciales) y C^* (conocimiento parcial) en la función cognitiva, obtenemos una versión “limitada” de la MCE en cada etapa de evolución. Denotemos por $\mathbb{S}^*$ el aproximador del espacio de soluciones, que crece a medida que $\mathbb{A}^*$ y C^* se expanden:

$$\mathbb{S}^* = \bigcup_{n=0}^{\infty} S_n^*, \quad S_n^* \subset \mathbb{S}, \quad S_0^* = \emptyset, \\ \text{y} \quad \lim_{n \rightarrow \infty} S_n^* = \mathbb{S}.$$

El sistema cognitivo parcial, en la etapa n , está representado por:

5.2. Técnica de Expansión $\mathbb{B}^*$

$$T : \mathbb{P} \times A_n^* \times C_n^* \longrightarrow S_n^*,$$

$$\forall p \in \mathbb{P}, \quad T(p, A_n^*, C_n^*) = \begin{cases} s \in S_n^* \subseteq \mathbb{S}, & \text{si existe } s \models p \text{ usando } A_n^*, C_n^*, \\ \emptyset, & \text{en caso contrario (no se encuentra solución).} \end{cases}$$

- Si p requiere alguna acción $a \notin A_n^*$, entonces T no puede construir s . En tal caso, se dice que la salida es el conjunto vacío $\emptyset$ (no hay solución alcanzable).
- Si p exige conocimiento que no está en C_n^* (i.e., $p \notin \text{alcance}(C_n^*)$), nuevamente T retornará $\emptyset$.
- Conforme n crece, $A_n^* \rightarrow \mathbb{A}$ y $C_n^* \rightarrow C$, de modo que $S_n^* \rightarrow \mathbb{S}$. En el límite, recuperamos la MCE ideal:

$$T : \mathbb{P} \times \mathbb{A} \times C \longrightarrow \mathbb{S}.$$

Con esto, hemos establecido una descripción rigurosa de cómo una MCE parcial—equipada con aproximadores de acciones y de conocimiento—puede converger hacia una MCE ideal en el infinito al incrementar sucesivamente A_n^* y C_n^* .

5.2 Técnica de Expansión $\mathbb{B}^*$

En la sección anterior vimos que una MCE ideal se alcanza solamente en el límite cuando los aproximadores $\mathbb{A}^*$ y C^* agregan infinitamente nuevas acciones y conocimiento, respectivamente. Sin embargo, desde un punto de vista práctico deseamos que la MCE sea capaz de encontrar soluciones para un gran número de problemas sin tener que incorporar todas las acciones de $\mathbb{A}$. Para ello introduciremos una técnica que denominamos *expansión $\mathbb{B}^*$* .

Sea $\mathbb{B}$ un conjunto finito de acciones con la siguiente propiedad:

$\forall a \in \mathbb{A}$, a puede expresarse como una composición finita de elementos en $\mathbb{B}$.

Más concretamente, existe una función de composición

$$\text{comp} : \bigcup_{n=1}^N \mathbb{B}^n \longrightarrow \mathbb{A},$$

5.2. Técnica de Expansión $\mathbb{B}^*$

tal que para cada $a \in \mathbb{A}$ existe alguna secuencia $(b_{i_1}, b_{i_2}, \dots, b_{i_k}) \in \mathbb{B}^k$ (con $k \leq N$) que satisface

$$\text{comp}(b_{i_1}, b_{i_2}, \dots, b_{i_k}) = a.$$

En entornos parcialmente controlados —por ejemplo, un ordenador— es habitual que $\mathbb{B}$ sea el conjunto finito de acciones de muy bajo nivel (como “pulsar tecla” y “mover ratón”), y dichas acciones basten para resolver cualquier problema $p \in \mathbb{P}$ en ese entorno.

Diremos que $\mathbb{B}$ es completo si, para cada problema $p \in \mathbb{P}$ existe al menos una solución

$$s = (a_1, a_2, \dots, a_n) \in \mathbb{S}_p$$

tal que cada a_i se puede descomponer en términos de acciones en $\mathbb{B}$. En consecuencia, cada problema p tiene al menos una solución compuesta exclusivamente por elementos de $\mathbb{B}$.

Hasta ahora también hemos asumido que todas las soluciones $s \in \mathbb{S}_p$ son equivalentes en tanto resuelven el mismo problema p . En la práctica, sin embargo, existe un criterio de *optimalidad* según algún parámetro (tiempo, recursos, longitud de la secuencia, etc.). Denotemos por “ $\succ$ ” la relación de orden en $\mathbb{S}_p$ tal que

$$s_i \succ s_j \iff s_i \text{ es más óptima que } s_j \text{ respecto a algún parámetro dado.}$$

Bajo la propiedad de completitud de $\mathbb{B}$, la MCE que solo dispone de $\mathbb{B}$ (y de un C^* parcial) generará siempre una *solución subóptima* para cada $p \in \mathbb{P}$, dado que no puede razonar con acciones de nivel más fino en $\mathbb{A}$.

Para mejorar la calidad de las soluciones (esto es, aproximarnos a las soluciones óptimas según $\succ$) sin renunciar a la garantía de encontrar alguna solución cuando p está en el alcance de C^* , definimos la técnica de *expansión* $\mathbb{B}^*$ de la siguiente forma:

$$\mathbb{B}^* = \mathbb{B} \cup \mathbb{A}^*,$$

Con esto, definimos una MCE expandida que opera con el espacio de acciones $\mathbb{B}^*$ y la base de conocimiento parcial C^* :

$$T : \mathbb{P} \times \mathbb{B}^* \times C^* \longrightarrow \mathbb{S}^*.$$

5.2. Técnica de Expansión $\mathbb{B}^*$

Para cada problema $p \in \mathbb{P}$, se tiene:

$$T(p, \mathbb{B}^*, C^*) = \begin{cases} s_{\text{sub}} \in S_0^* = \mathbb{S}_{\mathbb{B}}^*, & \text{si } p \in \text{alcance}(C^*) \quad (\text{solución subóptima}), \\ s_{\text{opt}} \in S_n^*, n > 0, s_{\text{opt}} \succ s_{\text{sub}}, & \text{si } \mathbb{A}^* \text{ ha crecido lo suficiente para incluir acciones} \\ & \text{que optimicen } s_{\text{sub}}. \end{cases}$$

En particular:

- Si $p \in \text{alcance}(C^*)$, existe siempre al menos una secuencia s_{sub} compuesta únicamente de acciones de $\mathbb{B}$. Así, $T_{\mathbb{B}^*}$ retorna una solución inicial subóptima.
- A medida que $\mathbb{A}^*$ incorpora nuevas acciones de $\mathbb{A}$, la MCE podrá generar secuencias s_{opt} tales que $s_{\text{opt}} \succ s_{\text{sub}}$. Es decir, la calidad de la solución mejora conforme crece $\mathbb{A}^*$.
- En el límite, cuando $\mathbb{A}^* \rightarrow \mathbb{A}$, recuperamos la MCE ideal:

$$T_{\text{ideal}} : \mathbb{P} \times \mathbb{A} \times C \longrightarrow \mathbb{S}, \quad T_{\text{ideal}}(p, \mathbb{A}, C) = s_{\text{opt}} \in \mathbb{S}_p,$$

donde s_{opt} es una solución óptima (o al menos factible) para p .

Este mecanismo nos otorga varias ventajas:

1. **Garantía de Solución Subóptima:** Si $p \in \text{alcance}(C^*)$, entonces siempre existe $s_{\text{sub}} \in \mathbb{S}_{\mathbb{B}}^*$ con $s_{\text{sub}} \models p$, sin necesidad de que $\mathbb{A}^*$ contenga todas las acciones de $\mathbb{A}$.
2. **Mejora Progresiva:** A medida que $\mathbb{A}^*$ crece, las nuevas acciones permiten *refinar* o *acortar* la secuencia s_{sub} , generando s_{mej} tales que $s_{\text{mej}} \succ s_{\text{sub}}$.
3. **Modularidad:** El comportamiento del sistema se divide en dos componentes: • Un conjunto finito $\mathbb{B}$ que asegura solución inicial. • Un aproximador $\mathbb{A}^*$ que mejora la óptima con el tiempo.
4. **Flexibilidad en la Base de Conocimiento:** Mientras C^* siga ampliándose, la técnica $\mathbb{B}^*$ garantiza que el sistema resolverá todos los problemas en el alcance, independientemente de la extensión actual de $\mathbb{A}^*$.

En resumen, la expansión $\mathbb{B}^*$ propone combinar un conjunto finito de acciones $\mathbb{B}$ con un aproximador creciente $\mathbb{A}^*$ para equilibrar la garantía mínima de resolución (vía $\mathbb{B}$) y la optimización progresiva de soluciones (vía $\mathbb{A}^*$).““

5.3 Máquina de Cognición-Ejecución

Hasta este punto hemos definido la MCE como una función capaz de seleccionar una secuencia de acciones que resuelva el problema $p \in \mathbb{P}$, bajo la suposición de que disponemos de una base de conocimiento parcial C^* y de un conjunto de acciones en B^* . Sin embargo, para modelar con precisión la arquitectura cognitiva real, conviene distinguir dos etapas conceptuales:

1. **Función de Cognición** (T_C): Identificación y generación de la *intención* de las acciones necesarias para resolver p , empleando la base de conocimiento parcial C^* .
2. **Función de Ejecución** (T_E): Transformación de cada acción simbólica en una interacción concreta con el entorno (o con el propio sistema), de modo que la intención generada se materialice efectivamente.

Para formalizar esta distinción, introducimos el *mapa de correspondencia* Λ , que relaciona cada acción simbólica con su ejecución real. Denotamos por $\lambda \in \Lambda$ el resultado de dicha correspondencia. Así, cada elemento λ describe el “modo de ejecución” de una acción concreta.

Definimos la función de cognición T_C como un mapeo entre problemas p y la intención de acciones C^* :

$$T_C : \mathbb{P} \times C^* \longrightarrow \Lambda,$$

tal que

$$T_C(p, C^*) = \lambda \iff \lambda = (\alpha_1, \alpha_2, \dots, \alpha_m),$$

donde cada α_i es un token simbólico en $\mathbb{A}^*$ que representa la *intención* de una acción necesaria para resolver p . En esta etapa, λ codifica *qué* acciones deben ejecutarse (por ejemplo, “hacer clic en el ícono de Firefox”), pero no realiza la interacción efectiva.

Definimos la función de ejecución T_E como un mapeo entre la intención λ y las acciones concretas B^* que se ejecutarán en el entorno:

$$T_E : \Lambda \times B^* \longrightarrow \mathbb{S},$$

donde, para $\lambda = (\alpha_1, \dots, \alpha_m)$:

$$T_E(\lambda, B^*) = s = (b_1, b_2, \dots, b_m) \in \mathbb{S}_p \in \mathbb{S}$$

5.3. Máquina de Cognición-Ejecución

Aquí, cada α_i se traduce a una acción concreta $b_i \in B^*$ mediante el mapa implícito en Λ . Si todas las acciones simbólicas α_i tienen su correspondiente $b_i \in B^*$, T_E construye la secuencia s que satisface p .

En este sentido, Λ es el *lenguaje intermedio* entre la capa de cognición y la de ejecución. Cada $\lambda = (\alpha_1, \dots, \alpha_m)$ con $\alpha_i \in \mathbb{A}^*$ se asocia unívocamente con una acción concreta $b_i \in B^*$. En la práctica, Λ se implementa como un módulo traductor que mapea tokens simbólicos a rutinas de ejecución concretas.

Finalmente, la MCE en su forma canónica se corresponde con la composición:

$$T_C(p, C^*) = \lambda \mid T_E(\lambda, \mathbb{B}^*) = s$$

Donde $s \models p$ siempre que $\mathbb{B}$ sea completo y λ se derive de C^* .

Ejemplo ilustrativo

Supongamos que p es el problema “Abrir Firefox”.

1. **Capa de Cognición.** Dado p y una base de conocimiento parcial C^* , la función de cognición T_C genera la intención λ :

$$T_C(\text{“Abrir Firefox”}, C^*) = \lambda = (\alpha_1) \rightarrow \alpha_1 = \text{“hacer clic en el ícono de Firefox”}.$$

2. **Capa de Ejecución.** Dado $\lambda = (\alpha_1)$, el traductor Λ asocia $\alpha_1 \mapsto b_1$, donde

$$b_1 = \text{“click(500, 900)”} \in \mathbb{B}^*.$$

Entonces,

$$T_E((\alpha_1), B^*) = s = (b_1).$$

Dado que $b_1 \in \mathbb{B}^*$, obtenemos $s \models p$ y Firefox se abre.

En los siguientes capítulos se describirá la implementación práctica de esta arquitectura, donde:

- C^* estará constituido por uno o varios modelos de lenguaje (LLMs) que aproximen la base de conocimiento C .

5.3. Máquina de Cognición-Ejecución

- $\mathbb{B}^*$ incluirá librerías y rutinas de automatización (por ejemplo, control de GUI, APIs del sistema operativo, scripts precompilados, etc.) para garantizar completitud.
- Λ se implementará como un módulo de traducción entre instrucciones simbólicas (en lenguaje natural o código intermedio) y las rutinas concretas de ejecución.

6. ARQUITECTURA SOFTWARE

En el presente capítulo se describe la arquitectura software empleada para la implementación de la MCE. Es importante resaltar que dicha arquitectura debe coordinar y optimizar múltiples componentes que interactúan entre sí, donde la forma en la que se integran estos componentes impacta directamente en la capacidad de la MCE para resolver problemas de manera eficiente y efectiva. Por ello, en este capítulo se presenta una arquitectura modular que facilita la experimentación y el reemplazo de componentes sin afectar al resto del sistema. Gracias a esta modularidad, el capítulo de experimentación podrá evaluar distintas aproximaciones cognitivas para la MCE sin necesidad de desarrollar un sistema independiente para cada una de ellas.

Con el fin de simplificar la lectura de los capítulos posteriores, a partir de ahora se entenderá por *MCE* a la arquitectura software descrita en este capítulo. Cuando se haga referencia al fundamento teórico de la MCE, se empleará el término *modelo teórico* de la MCE.

6.1 Capas y Módulos de la MCE

La MCE desarrollada está compuesta por cuatro módulos principales, cada uno directamente relacionado con las entidades definidas en el modelo teórico de la MCE. Estos módulos son:

- **Capa de Cognición:** Este módulo contiene la lógica cognitiva de la MCE. Aquí se integran los algoritmos de percepción, aprendizaje, comportamiento, planificación y razonamiento de los agentes de la MCE. Cada agente utiliza la información obtenida de las demás capas para tomar decisiones e interactuar con su entorno de manera específica. No obstante, todos ellos comparten la dependencia de uno o varios modelos de lenguaje para la toma de decisiones.

Esta capa se corresponde directamente con la función de cognición del modelo teórico de la MCE, donde la base de conocimiento $\mathbb{C}$ equivale a la información extraída de los LLMs y el mapa de correspondencia Λ resulta de

6.1. Capas y Módulos de la MCE

la conversión de dicho razonamiento en tokens que, posteriormente, serán procesados por la Capa de Ejecución.

- **Capa de Ejecución:** Este módulo es el responsable de traducir los tokens generados por la Capa de Cognición en un conjunto de acciones que la MCE ejecutará. En capítulos posteriores se describirá este proceso como “interpretación” del plan generado por la LLM. Más allá de una simple concatenación de acciones, la Capa de Ejecución gestiona la creación de procesos, la desambiguación de tokens, la paralelización, la trazabilidad y la gestión de errores a nivel software.

Al igual que la Capa de Cognición, esta capa se relaciona directamente con la función de ejecución del modelo teórico de la MCE, al traducir tokens Λ en acciones pertenecientes al espacio $\mathbb{B}^*$.

- **Espacio de Acciones:** Este módulo contiene el conjunto finito de acciones que la MCE puede ejecutar. Incluye tanto las acciones elementales del espacio $\mathbb{B}$ del modelo teórico de la MCE como un conjunto creciente de acciones correspondientes al aproximador $\mathbb{A}^*$. Entre las acciones elementales se encuentran mecanismos para la percepción de elementos en pantalla e interacción con el entorno vía mouse o teclado, mientras que las acciones de $\mathbb{A}^*$ abarcan operaciones de alto nivel e integración con APIs del sistema operativo.

Como se detallará más adelante, la MCE puede dividirse en dos máquinas: una máquina maestra que ejecuta todas las capas de la MCE y una máquina esclava que realiza las acciones recibidas del maestro. Por esta razón, el Espacio de Acciones es compatible con diversos sistemas operativos (por ejemplo, Windows, Linux/Ubuntu o Raspbian), de modo que cualquiera de ellos puede actuar como esclavo de una MCE remota.

- **Capa Núcleo:** Este módulo se encarga de orquestar y coordinar todos los módulos de la MCE, además de construir los recursos necesarios para su funcionamiento. A diferencia de los demás módulos, la Capa Núcleo no está diseñada para ser reemplazada ni forma parte del modelo teórico de la MCE. En cambio, su función es aprovechar las capacidades de las demás capas para ejecutar la MCE, ya sea de forma local o remota.

Adicionalmente a estos módulos, la MCE incluye un conjunto de herramientas como el registro de entidades, instaladores o manejadores del sistema que ayudan en la coordinación de todos las capas y módulos.

En las siguientes secciones se describirá con mayor detalle cada uno de los módulos y herramientas de la MCE. La descripción se realizará comenzando desde las interacciones a más bajo nivel, es decir, desde el propio espacio de acciones, y se irán conformando abstracciones de alto nivel a medida que nos acercamos a las capas de cognición.

6.2 Espacio de Acciones

Dentro de esta capa debemos integrar acciones que satisfagan las propiedades definidas en el espacio de acciones $\mathbb{B}^*$ del modelo teórico de la MCE. Todas estas acciones serán ejecutadas en la máquina controlada, a la que denominaremos como máquina esclava. Estas acciones se dividen en dos grupos, el espacio de acciones elementales $\mathbb{B}$ y el espacio de acciones $\mathbb{A}^*$.

A su vez, el espacio de acciones también puede ser categorizado en 3 distintas categorías:

- **Acciones actuadoras:** Son aquellas que interactúan con el entorno en el se encuentra la máquina controlada. Estas permiten al agente cambiar el estado actual y la composición de este tipo de acciones permite alcanzar el objetivo final. No es posible completar en su totalidad un objetivo sin la ejecución de al menos una acción actuadora.
- **Acciones perceptivas:** Son aquellas que permiten al agente obtener información de su entorno. Estas acciones son fundamentales para la toma de decisiones, pues no será posible alcanzar muchos objetivos sin previamente conocer el estado actual de la máquina esclava.
- **Meta-Acciones:** Son aquellas que permiten al agente modificarse a si mismo, ya sea para guardar razonamientos en memoria, para activar distintos modos de razonamiento o para cambiar el estado del agente. Esto permite al agente adaptarse a entornos cambiantes.

A continuación se describen algunas de las acciones más relevantes de la MCE. A su vez, categorizaremos cada una de ellas según su tipo y espacio de acciones al que pertenecen.

6.2.1 Acciones Elementales

Aunque el espacio de acciones $\mathbb{B}$ puede ser discutido y ampliado con el tiempo, a continuación se realizará un resumen de la implementación de las acciones más fundamentales para una MCE en el entorno de un escritorio clásico. Estas son, el control del teclado, la captura de pantalla y el control del mouse.

Control del Teclado

Las acciones de control del teclado son sencillas de implementar dadas las librerías `pyautogui` y `uinput` de Python. En el primero caso la función `write()` de `pyautogui` realiza la escritura de un texto dado de forma automática.

En el caso de usar un sistema operativo Linux o Raspbian, la librería `uinput` nos ofrece un control mas granular sobre los eventos del teclado, donde `pyautogui` puede fallar debido a problemas de incompatiibilidad o permisos con distintas distribciones de Linux. En este caso, resulta necesario realizar un remapeo entre las los posibles eventos de teclado (`uinput.KEY_A`, `uinput.KEY_MINUS`, etc.) y los caracteres de la cadena de texto objetivo a escribir ('a', '-', etc.).

Esta funcionalidad, se envolverá dentro de una función unica para todos los sistemas operativos, y permitira al agente escribir texto en la máquina esclava de forma autónoma:

```
>> write("Este texto será escrito usando el teclado")
>> Este texto será escrito usando el teclado
```

Un caso de uso un poco más complejo es el de utilizar combinaciones de teclas o atajos de teclado. En este caso, podemos utilizar las funciones de `pyautogui` o `uinput` para enviar una secuencia de teclas, sin embargo, necesitamos definir un estándar para la representación de estas combinaciones. Para resolver esto, se ha creado un diccionario de combinaciones de teclas que incluyen multiples representaciones de una misma combinacion o tecla, como puede ser `CTRL`, `CONTROL` o `CTRL_LEFT`. De esta forma el agente podría utilizar la siguiente sintaxis:

```
>> press("CTRL + C")
KeyboardInterrupt
```


Estos dos tipos de acciones, entran en la categoría de acciones actuadoras y pertenecen al espacio de acciones elementales $\mathbb{B}$. Esto se debe a que multiples aplicaciones no pueden ser controladas sin la capacidad de escribir texto o enviar combinaciones de teclas.

Captura de Pantalla

La captura de pantalla es una de las acciones perceptivas que más información brinda sobre el estado actual de la máquina esclava. Sin embargo, a su vez esta información tiene el coste computacional más alto, ya que requiere tanto un pre-procesamiento de la imagen como un análisis posterior para extraer la información necesaria.

Para la captura de pantalla se utiliza la librería `pyautogui`, que atravesará por varios modulos de OCR (Reconocimiento Óptico de Caracteres), algoritmos de detección de elementos (creados con `OpenCV`)¹ y modelos multimodales basados en LLM para extraer tanto las entidades visuales como textuales de la imagen capturada. Una vez procesada la imagen, se puede generar una representación de la misma mediante un conjunto de bounding boxes que indican la posición de los elementos detectados y que, posteriormente, será utilizada para la toma de decisiones.

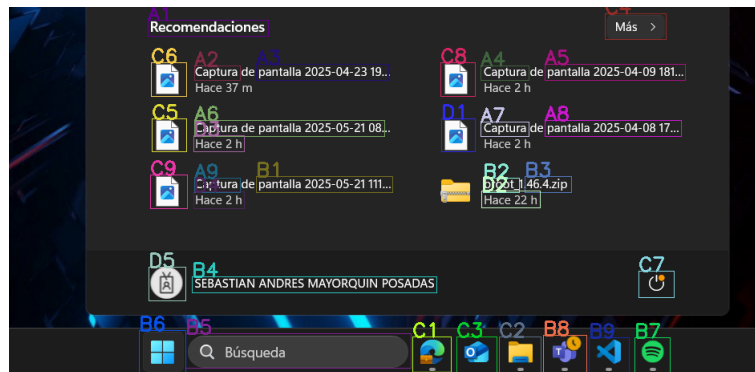


Figure 6.1: Procesamiento de una captura de pantalla con algoritmos de detección de elementos. Fuente: Elaboración Propia.

Notese que aunque esta acción es perceptiva, tambien forma parte de las acciones elementales $\mathbb{B}$ del espacio de acciones, ya que multiples problemas no pueden

¹Adicionalmente para eliminar los falsos positivos, se utiliza una métrica basada en Iou (Intersection over Union)

ser resueltos sin distinguir los elementos presentes de forma visual, como por ejemplo, la interfaz gráfica de un videojuego.

Control del Mouse

El control del mouse es una acción actuadora y elemental que permite al agente interactuar con la interfaz gráfica de la máquina esclava. Para llevarla a cabo, se puede utilizar la librería `pyautogui` tanto en Windows como en Raspbian.

En sistemas Linux sin embargo, es importante considerar el tipo de entorno gráfico utilizado. En entornos basados en X11, los permisos para controlar el mouse son relativamente permisivos. En cambio en sistemas que utilizan Wayland, por motivos de seguridad, los eventos del mouse solo pueden enviarse a la ventana que esté activa originalmente.

Esto significa que, si el agente intenta controlar una ventana distinta, el movimiento del mouse quedará bloqueado hasta que el usuario vuelva a activar manualmente la ventana original.

Estos problemas de compatibilidad se pueden resolver utilizando herramientas como `ydotool` o bien realizando una calibración previa de la sensibilidad del mouse y usando `uinput` para enviar eventos de mouse.

Una vez resueltos estos problemas, el agente podrá interactuar con la máquina tal que:

```
>> move_mouse(100, 200)
>> double_click()
>> move_mouse(300, 400)
>> right_click()
```

6.2.2 Espacio de Acciones $\mathbb{A}^*$

El espacio de acciones $\mathbb{A}^*$ es infinito por definición. En el caso de la arquitectura implementada se han desarrollado alrededor de 20 paquetes de acciones, donde cada uno de estos paquetes contiene entre 1 y 3 acciones dependiendo de la complejidad del problema a resolver.

A continuación se describen algunas de las acciones más relevantes del espacio de acciones $\mathbb{A}^*$:

Espera Condicional

La espera condicional o `wait_for()` es una meta-acción que permite al agente esperar hasta que se cumpla una determinada condición antes de continuar con la ejecución de la siguiente acción. Esta acción permite ahorrar una gran parte de los recursos computacionales gastados mientras un elemento se carga o una ventana se abre.

Para dejar al agente bloqueado, bastará con utilizar la función `sleep()` de Python, sin embargo la detección de la condición a esperar es una tarea más sensible y compleja. Para ello, se utiliza un modelo multimodal que es capaz de recibir una condición dada en lenguaje natural, y analizará periódicamente la pantalla para determinar si la condición se ha cumplido o no.

Como ejemplo, el agente podría esperar a que aparezca un botón de Aceptar en la pantalla tal que:

Es importante tener en cuenta los casos en los que la condición nunca se cumple, ya que esto podría bloquear al agente indefinidamente. Para evitarlo, el propio agente puede establecer un tiempo máximo de espera, tras el cual se cancelará la espera y se continuará con la ejecución del programa.

```
>> wait_for("Aparición del botón 'Aceptar'", timeout=30)
Esperando a que aparezca el botón Aceptar...
Esperando a que aparezca el botón Aceptar...
Esperando a que aparezca el botón Aceptar...
Condición cumplida: El botón Aceptar ha aparecido en la pantalla.
>>
```

Abrir Aplicación

La acción de abrir una aplicación es una de las más comunes y útiles para la MCE, pues una gran parte de los problemas a resolver requieren la interacción con aplicaciones específicas. Así pues esta acción permite traducir lenguaje natural a una búsqueda de aplicaciones en el sistema operativo esclavo.

6.2. Espacio de Acciones

Para realizar esto, se realiza una búsqueda en múltiples directorios con usualmente utilizados como directorios de instalación de aplicaciones, como por ejemplo, /usr/bin en el caso de Linux o C:\Program Files en el caso de Windows; y se utiliza un algoritmo de similitud de cadenas para encontrar la aplicación más similar al nombre de la aplicación proporcionado por el agente.

Gracias a esta acción, el agente puede abrir e interactuar con aplicaciones tal que:

```
>> open_application("Spotify")  
Abriendo Spotify...  
>>
```

Nótese que aunque esta acción puede ser completada unicamente mediante acciones elementales (clicks y escritura de texto), su implementación permite una interacción mas agil y eficiente con el sistema. Formalmente, reemplazamos una solución s_{sub} con una solución s_{opt} ambas pertenecientes a $\mathbb{S}_p$

Ejecución de Código

La acción de ejecución de código permite al agente ejecutar código Python, JavaScript, bash o powershell en la máquina esclava. Esta acción tiene optimiza especialmente las tareas de gestión y/o modificación de archivos, ya que permite al agente realizar operaciones complejas de forma rápida y eficiente.

Sin embargo, es importante tener en cuenta que esta acción puede ser peligrosa si el agente no esta debidamente controlado, es por ello que se ha implementado adicionalmente una interfaz gráfica de seguridad que mantiene al usuario informado del código que se está ejecutando y le permite garantizar o denegar la ejecución de la acción en cualquier momento.

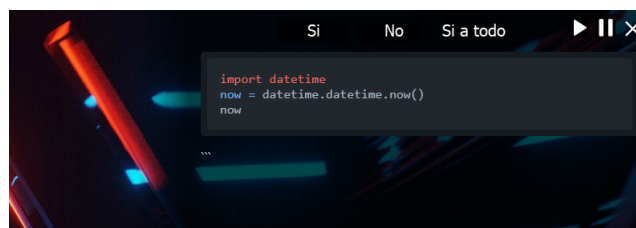


Figure 6.2: Interfaz gráfica de seguridad para ejecución de código. Elaboración Propia

6.3 Capa de Ejecución

Como se ha descrito en secciones anteriores, la capa de ejecución es la encargada de traducir los tokens generados por la capa de cognición en acciones que la MCE ejecutará en la máquina esclava.

Este proceso de ejecución utiliza 2 componentes principales:

- **Exelent:** Exelent es un lenguaje declarativo de alto nivel concebido y diseñado en este proyecto para ser utilizado por modelos de lenguaje. Este lenguaje viene junto a un mecanismo de procesamiento que tokeniza/compila este lenguaje y lo convierte en una representación que puede ser entendida por un intérprete.
- **Intérprete:** El intérprete es el componente encargado de enlazar los tokens generados por la capa de Cognición con las acciones correspondientes en el espacio de acciones $\mathbb{B}^*$. A su vez, el intérprete convierte los tokens en "tareas" que pueden ser ejecutadas. Estas tareas siempre se asegura que estan definidas siguiendo el protocolo TSP (Task-Sequence Protocol), que define el conjunto de reglas que ambos componentes deben seguir para garantizar la correcta sincronización de los componentes.

A continuación se describen con más detalle los mecanismos internos de cada uno de estos componente y su funcionamiento en la capa de ejecución.

6.3.1 Task-Sequence Protocol (TSP)

El protocolo TSP es un conjunto de reglas que define cómo deben ejecutarse las acciones en la MCE. Este protocolo contiene una estructura de 3 niveles que permite organizar las acciones de forma jerárquica.

Estos 3 niveles son:

- **Acción:** Una acción es la unidad más básica de ejecución en el protocolo TSP. Cada acción representa una única acción correspondiente al espacio de acciones $\mathbb{B}^*$. Puesto que todas las acciones se representan a su vez mediante una única funcion Python, cada acción estará compuesta por un nombre de

6.3. Capa de Ejecución

la acción y un conjunto de argumentos que serán utilizados por la función correspondiente.

- **Secuencia:** Una secuencia es un elemento que agrupa acciones y/o otras secuencias y define un mecanismo de ejecución. Este mecanismo puede ser secuencial, paralelo, condicional, iterativo, etc. Las secuencias y/o acciones serán ejecutadas siguiendo el mecanismo definido por la secuencia. Esto permite al agente combinar acciones de forma flexible y adaptativa.
- **Tarea:** Una tarea es el elemento más alto del protocolo TSP y representa una unidad de trabajo que puede ser ejecutada por la MCE. Una tarea puede (de forma opcional) contener una prioridad, un identificador y modificadores de control.

La prioridad permite al agente crear situaciones en las que ciertas acciones deben ser ejecutadas interrumpiendo otras acciones en curso; por ejemplo, un agente puede decidir parar un análisis si recibe una notificación de un evento importante.

El identificador es generado automáticamente por el intérprete y permite a la capa de ejecución mantener un registro de las tareas en curso.

Los modificadores de control permiten al agente definir condiciones de ejecución como "esta tarea debe ejecutarse solo si la tarea anterior ha finalizado con éxito". Estos modificadores son especialmente útiles para la creación de tareas complejas que requieren de una ejecución controlada y secuencial.

En el caso del lenguaje exelent, este está estructurado de forma que todas las tareas contienen los tres niveles del protocolo TSP. A su vez, el intérprete debe ser capaz de convertir cualquier tarea que siga el protocolo TSP en código ejecutable por la MCE.

6.3.2 Exelent

Exelent (haciendo referencia al juego de palabras "Exe-cution Lan-guage") es un lenguaje declarativo que permite a los modelos de lenguaje escribir acciones siguiendo el protocolo TSP utilizando lenguaje natural. A su vez, este lenguaje está diseñado para poder ser generado por LLMs sin necesidad de realizar fine-tuning o entrenamiento adicional. Esto se debe a su sencillez de estructura y gramática lo que permite su utilización con distintos modelos de forma efectiva.

6.3. Capa de Ejecución

A modo de ejemplo conceptual, se presenta a continuación un fragmento de pseudocódigo que representa la lógica general de exelent sin exponer detalles de implementación concretos:

```
>> start opening firefox with priority normal
>>   parallel run
>>     open_application "Firefox"
>>     wait_for "Firefox is ready"
>>
>>   on exit
>>     if "Firefox is not ready" then
>>       replan "Firefox failed to open"
>>     else
>>       complete "Firefox opened successfully"
>>     end if
>>
>>   on error
>>     replan "An error occurred while opening Firefox"
>>
```

En este ejemplo, el agente inicia la apertura de Firefox con una prioridad normal. Dentro de esta tarea, vemos como distintos tipos de secuencia permiten definir la lógica de ejecución. Por ejemplo, la secuencia `parallel run` permite ejecutar dos acciones de forma simultánea. A su vez estas secuencias contienen acciones que se ejecutarán siguiendo el protocolo TSP.

Para poder tokenizar este lenguaje, se ha desarrollado un tokenizador que convierte cada línea de código en estructuras de datos que serán utilizados por el intérprete. A continuación, se muestra una tokenización hipotética del código, útil para entender la estructura del formato.

```
>> {
>>   "task": {
>>     "name": "opening firefox",
>>     "priority": "normal",
>>     "sequences": [
>>       {
>>         "type": "parallel",
>>         "actions": [
```

```

>>         {"action": "open_application", "args": ["Firefox"]},
>>         {"action": "wait_for", "args": ["Firefox is ready"]}
>>     ]
>> }
>> ],
>> "on_exit": {
>>     "if": {
>>         "condition": "Firefox is not ready",
>>         "then": {"action": "replan", "args": ["Firefox failed to open
>>         "else": {"action": "complete", "args": ["Firefox opened succe
>>     }
>> },
>> "on_error": {"action": "replan", "args": ["An error occurred while op
>> }
>> }

```

6.3.3 Intérprete

El intérprete es el componente encargado de procesar los tokens generados desde el lenguaje exelent tokenizado y crear una estructura de datos ejecutable que cumpla las condiciones del protocolo TSP. A su vez el intérprete es el encargado de enlazar las acciones con las funciones correspondientes del espacio de acciones $\mathbb{B}^*$.

En este proyecto se han implementado dos intérpretes distintos², uno basado en servicios del framework ROS2 (denominado como ROSA) que permitió la creación de un prototipo funcional y depurable; y otro creado puramente en Python (denominado como Pyxcel, del juego de palabras "Py-thon E-xcel-ent") que realiza enlaza las acciones *just-in-time*, es decir, cargando los módulos necesarios únicamente cuando son necesarios.

Pyxcel es un intérprete mas eficiente, ligero y modular que ROSA, eliminando complejidades innecesarias (debido principalmente a la naturaleza modular de ROS2) y permitiendo una integración más sencilla con el resto de la MCE.

Para realizar el enlace de las acciones, el intérprete consulta un mecanismo global de la MCE denominado como ItemRegistry o *Registro de Entidades*. Este reg-

²Nótese que la naturaleza modular de la MCE permite que el intérprete pueda ser reemplazado por cualquier otro intérprete que cumpla las condiciones del protocolo TSP.

istro (que será descrito en más profundidad en secciones posteriores) contiene una base de datos de todas las acciones disponibles en el espacio de acciones $\mathbb{B}^*$ y el código Python correspondiente a cada una de ellas. De esta forma, el intérprete consulta el registro para obtener todas las acciones similares a la acción solicitada y realiza un proceso de desambiguación para seleccionar la acción más adecuada.

Una vez se ha seleccionado la acción, el intérprete crea un proceso paralelo y vigila su ejecución. Esta acción puede a su vez generar fragmentos de retroalimentación que recibidos por el intérprete, serán procesados y enviados a la capa de cognición. Mientras la ejecución se encuentra en curso, el intérprete también puede recibir nuevos eventos desde la capa de cognición, donde el intérprete puede decidir si continuar con la ejecución, mandar señales de parada, cancelación o realizar un envío de recursos adicionales.

Gracias a este mecanismo, el resto de capas de la MCE pueden utilizar lenguaje Exelent para poner en marcha la ejecución de una tarea, sin necesidad de preocuparse por los detalles de control o implementación de cada acción.

6.4 Capa de Cognición

La capa de cognición es el conjunto de componentes que permiten dotar de "inteligencia" cognitiva a la MCE. Formalmente, esta capa se corresponde con la función de cognición del modelo teórico de la MCE. Nótese que en esta capa asumiremos que la base de conocimiento $\mathbb{C}$ es un modelo de lenguaje natural.

Así pues esta capa contiene tanto un kit de herramientas agénticas que dotarán al sistema de capacidades cognitivas, como un conjunto de agentes que, utilizando estas herramientas, serán capaces de integrarse en la MCE y consecuentemente, resolver problemas de forma autónoma.

En este capítulo se describirán algunos de los mecanismos internos más relevantes de la capa de cognición independientemente de los agentes que los utilicen. Estos agentes serán descritos en el capítulo de experimentación, donde se presentarán distintas aproximaciones cognitivas para la MCE.

6.4.1 Autodescriptor de Acciones

Para que un agente pueda utilizar el espacio de acciones $\mathbb{B}^*$ de la MCE, es necesario que se cumplan las siguientes condiciones:

1. El agente debe ser previamente informado de las acciones disponibles en el espacio de acciones $\mathbb{B}^*$.
2. El agente debe ser informado de los requisitos o entradas de cada una de las acciones disponibles en el espacio de acciones, así como de los posibles resultados o salidas de cada una de ellas.
3. El agente debe conocer un formato de representación adecuado para poder utilizar el espacio de acciones de forma efectiva.

Para resolver estos 3 problemas, se ha desarrollado un mecanismo denominado como *Autodescriptor de Acciones*. Este mecanismo permite generar una descripción automática de las acciones disponibles en el espacio de acciones $\mathbb{B}^*$ de la MCE.

- **Autoformato:** El autodescriptor utiliza el *registro de entidades* de la MCE para obtener todas las acciones disponibles en el espacio de acciones. Para cada una de ellas utiliza un formato json que contiene el nombre de la acción, una breve descripción de su caso de uso, los argumentos necesarios para su ejecución y una breve descripción de los posibles resultados.
- **Mapa de Correspondencia:** Para cada una de las acciones, el autodescriptor enlaza el nombre de la acción con un conjunto de sinónimos y términos relacionados. Todos estos términos son correlacionados con la acción y se almacenarán en el mapa de correspondencia Λ utilizado en la capa de ejecución. De esta forma, cuando el agente solicite una acción, la capa de cognición podrá buscar en el mapa de correspondencia Λ y generar código excelente que pueda ser ejecutado por la capa de ejecución.
A su vez el autodescriptor también genera un set de instrucciones que pueden ser utilizadas por el agente para comprender cómo utilizar el espacio de acciones $\mathbb{B}^*$ de la MCE.

Como ejemplo ilustrativo, supongamos un espacio de acciones conformado exclusivamente por la acción de control de teclado `write()`. El autodescriptor generaría la siguiente descripción:

6.4. Capa de Cognición

```
>> {
>>   "name": "write",
>>   "description": "Escribe un texto dado en la máquina esclava.",
>>   "args": {
>>     "text": "El texto a escribir."
>>   },
>>   "returns": {
>>     "success": "Indica si la escritura fue exitosa."
>>   }
>> }
```

A su vez, el mapa de correspondencia Λ sería actualizado tal que:

```
>> {
>>   "write": [
>>     "write",
>>     "escribir",
>>     "introducir texto",
>>     "input"
>>   ]
>> }
```

De esta forma, el agente puede utilizar el espacio de acciones $\mathbb{B}^*$ de la MCE de forma efectiva, generando código exelent siguiendo las instrucciones embebidas en el autodescriptor.

```
>> agent.invoke("Por favor escribe 'Hola Mundo' en la máquina esclava")
>> Resultado:
>> '''
>> start writing hello world with priority normal
>>     sequentially run
>>         write "Hello World"
>> '''
```

Posteriormente, este fragmento de código sería enviado a la capa de ejecución para ser procesado por el intérprete y ejecutado en la máquina esclava.

Gracias a este mecanismo los agentes de la MCE podrán utilizar el espacio de acciones $\mathbb{B}^*$ de forma efectiva, sin necesidad de preocuparse por los detalles de la implementación.

6.4.2 Buffer de Memoria

Siguiendo la arquitectura PMPA discutida en la sección 2.5 la MCE implementa un buffer de memoria que permite a los agentes mantener coherencia a lo largo de la ejecución de una tarea.

En su forma más básica, una memoria no es más que un vector de mensajes que contienen tanto la entrada del usuario, como información del entorno, y los resultados de las acciones realizadas por el agente. Sin embargo, a nivel práctico la memoria debe contener únicamente la información relevante que el agente necesita para tomar decisiones y resolver problemas.

Para ello, la MCE implementa un buffer de memoria que en cada actualización realiza un análisis de la información almacenada, eliminando la información redundante, de gran volumen, o que no es relevante para la tarea en curso.

Como ejemplo ilustrativo, supongamos que un agente ha recibido la siguiente información en su buffer de memoria:

```
>> [
>>   ("user", "Por favor escribe 'Hola Mundo' en la máquina esclava
>>     y luego haz click en cerrar pestaña"),
>>   ("environment", "La máquina esclava está en el escritorio"),
>>   ("action", "start writing hello world with priority normal"),
>>   ("action", "write 'Hello World'"),
>>   ("result", "Escritura exitosa")
>> ]
```

En este caso, el buffer de memoria podría eliminar algunas entradas que ya no son relevantes para las proximas tareas. Así, el buffer de memoria podría actualizarse a:

```
>> [
>>   ("user", "Por favor escribe 'Hola Mundo' en la máquina esclava
```

```
>>         y luego haz click en cerrar pestaña"),
>>     ("environment", "La máquina esclava está en el escritorio"),
>>     ("result", "Escritura exitosa")
>> ]
```

Adicionalmente, el buffer de memoria también contiene mecanismos para minimizar el volumen de información gráfica (como capturas de pantalla) o generación automática de resúmenes para salidas con gran volumen de texto. Con todos estos mecanismos, los agentes mantendrán un estado coherente a lo largo de la ejecución de una tarea, sin necesidad de preocuparse por la gestión de la memoria.

6.4.3 Estado Cognitivo

El estado cognitivo es un mecanismo que permite a los agentes el uso de las meta-acciones. De forma simplificada, un estado cognitivo no es más que un conjunto de variables que definen el comportamiento que debe seguir el agente. Cuando un agente realiza una meta-acción, esta modifica su estado cognitivo, permitiéndole influir en su propio comportamiento.

Nótese como estas *variables* en el estado cognitivo no tienen una funcionalidad específica ni afectan en modo alguno a la MCE, sino que son utilizadas por el propio agente como indicadores de su comportamiento.

Como ejemplo ilustrativo, supongamos que un agente comienza una tarea con el siguiente estado cognitivo:

```
>> {
>>     "state": "normal",
>>     "short-term-goal": "None",
>>     "long-term-goal": "Play a song on Spotify"
>> }
```

En este caso, el agente podría cambiar su estado cognitivo a un modo de planificación utilizando la siguiente meta-acción:

```
>> start thinking with priority normal
>>     parallel run
```

```
>> set_state "planning"
>> set_short_term_goal "Generate a plan to play a song on Spotify"
```

De este modo, el agente habría cambiado su estado cognitivo a:

```
>> {
>>   "state": "planning",
>>   "short-term-goal": "Generate a plan to play a song on Spotify",
>>   "long-term-goal": "Play a song on Spotify"
>> }
```

Así pues, el estado cognitivo no necesita alterar directamente los mecanismos internos del agente, sino que su influencia se limita a modificar las instrucciones que el propio agente recibe.

De forma practica, si por ejemplo, el agente recibe un estado cognitivo que le indica que previamente él mismo se ha asignado un objetivo a corto plazo, este seguirá su propio protocolo de ejecución para alcanzar dicho objetivo, cambiando su estado nuevamente cuando lo considere apropiado.

El estado cognitivo a su vez, tambien puede ser programado de forma externa para modificarse automáticamente en función de eventos o condiciones del entorno. Por ejemplo, un agente, podria cambiar su estado cognitivo a "razonamiento rápido" si llevara más de 5 minutos realizando una tarea.

Adicionalmente, una de las meta-acciones mas importantes de la MCE es la capacidad para establecer el estado de "objetivo completado". Esta meta-acción será vigilada por la MCE para determinar si el agente ha completado su tarea o no. Mientras que el agente no complete su objetivo, la MCE continuará ejecutando las acciones enviadas a la capa de ejecución. Finalmente, cuando el agente complete su objetivo, la MCE recuperará el control y decidirá si continuar con la ejecución de la tarea o finalizarla.

6.4.4 Fast Agent-Protocol (FastAP)

El protocolo FastAP esta basado en el protocolo de agentes discutido en la sección 2.5. En contraste con el protocolo de agentes original, FastAP es un protocolo mucho más rápido y con una implementación sencilla que permite embeber todas las capacidades de un agente en una única funcion de Python.

A su vez, este protocolo es totalmente síncrono y basado en un sistema de mensajes, donde al enviar un mensaje a un agente que utiliza el protocolo FastAP, este tomara el control de la MCE de forma temporal hasta que haya completado su tarea.

Utilizando este protocolo todos los agentes de la MCE, independientemente de su implementación y mecanismos internos, pueden ser integrados en la MCE con una interfaz unificada.

A su vez, FastAP esta sincronizado con el resto de capas de la MCE para realizar las siguientes funcionalidades:

- **Resolución automática de dependencias:** FastAP realiza un análisis dinámico de las dependencias de cada agente, tanto a nivel de librerías como a nivel de las acciones necesarias para cada agente. Una vez cone estas dependencias, FastAP generará un entorno virtual que contiene todas las dependencias necesarias para la ejecución del agente. De esta forma, los agentes pueden ser ejecutados sin tener conflictos con el resto de agentes o capas de la MCE.
- **Control de Ejecución:** FastAP permite pausar, reanudar o cancelar la ejecución de un agente en cualquier momento. Esto es debido, a que todos los agentes necesitan que FastAP envíe mensajes de confirmación para continuar con la ejecución de su tarea. Cuando una señal de parada es enviada, FastAP no envía más mensajes de confirmación al agente y, en caso de que una posterior cancelación sea enviada, FastAP eliminará el entorno virtual del agente y devolverá el control a la MCE.
- **Logging y Depuración:** FastAP permite registrar todas las acciones realizadas por el agente independiente de que este utilice acciones, meta-acciones o que llame a otros agentes. Todas esta información puede ser enviada a un fichero de log o a cualquier sistema de depuración que sea especificado por el sistema.

Este protocolo es un mecanismo fundamental en la MCE, pues sin el no sería posible la experimentación y contraste de distintos agentes en la MCE. A su vez, este protocolo permite a los agentes integrarse de forma sencilla y rápida en la MCE, pues provee de una plantilla de código que puede ser facilmente adaptada a los mecanismos internos de cada agente.

6.5 Capa de Núcleo

La capa de núcleo tiene la responsabilidad de conectar todos los componentes de la MCE y coordinar su funcionamiento. El modelo teórico de la MCE, no contempla ninguna función específica para esta capa, sin embargo en la práctica, esta capa es la que permite la correcta comunicación entre las distintas capas y componentes de la MCE.

En este capítulo se describirán algunos de los mecanismos internos más relevantes de la capa de núcleo, finalmente, se presentará una visión general del funcionamiento de la MCE.

6.5.1 Registro de Entidades

El registro de entidades es un mecanismo que permite a la MCE almacenar todas las acciones del espacio de acciones $\mathbb{B}^*$, sus metadatos asociados y crear una interfaz estandarizada para que todas las capas puedan interactuar con estas acciones.

Entre las funcionalidades más relevantes del registro de entidades se encuentran:

- **Resolución de Dependencias:** Mientras que FastAP se encarga de resolver las dependencias de los agentes, el registro de entidades se encarga de resolver las dependencias de las acciones. Esto incluye tanto las dependencias de librerías como interdependencias entre acciones. Por ejemplo, si una acción de reconocimiento de caracteres (OCR) depende de la librería `pytesseract`, el registro de entidades se encargará de asegurar que esta librería esté disponible en un entorno virtual que luego será utilizado por la capa de ejecución.
- **Carga Perezosa:** El registro permite la carga perezosa de acciones, es decir, las acciones no se cargan hasta que son llamadas por la capa de ejecución. Esto permite que acciones que no son utilizadas no consuman recursos innecesarios en el sistema. Este mecanismo resulta fundamental dado que el espacio de acciones $\mathbb{B}^*$ es teóricamente tendiente a infinito, por lo que en caso de que la carga no fuera perezosa, el sistema podría consumir una cantidad de recursos computacionales excesiva tanto en el inicio de la MCE como en la ejecución de las acciones.

- **Mapa de Correspondencia:** El registro guarda un mapa de correspondencia Λ basado en múltiples diccionarios clave-valor que permiten a la capa de ejecución encontrar las acciones correspondientes a los tokens encontrados durante el análisis del lenguaje exelent. Debido a la carga perezosa, este mapa no apunta directamente a las funcionalidades de las acciones, sino que apunta a un disparador que, al ser invocado, cargará la acción en memoria y la ejecutará. En caso de haber sido cargada previamente, el disparador simplemente ejecutará la acción sin necesidad de cargarla de nuevo.
- **Entorno Maestro-Esclavo:** El registro permite crear un entorno maestro-esclavo para dividir la ECM en dos máquinas: una máquina maestra que sera la encargada de ejecutar la MCE y otra esclava que ejecutará las acciones del espacio de acciones. Para realizar esto, el registro es capaz de modificar el disparador de las acciones para que, en lugar de ejecutar la acción directamente, envíe un mensaje a la máquina esclava para que esta ejecute la acción en su lugar. A su vez, el registro evita que la acción se cargue en la máquina maestra, de forma que esta no consuma recursos innecesarios.
- **Almacenamiento de Configuraciones:** El registro tambien permite almacenar configuraciones que pueden ser utilizadas por distintas capas de la MCE. De esta forma, todas las capas pueden interactuar entre sin depender de la integración de cada una de ellas. A su vez, también permite al cliente cambiar el comportamiento de la MCE sin necesidad de modificar el código de cada una de las capas.
- **Empaquetado de Acciones:** El registro permite a su vez empaquetar conjuntos de acciones en un único paquete que comparten funcionalidades comunes. Estos paquetes agilizan la resolucion de dependencias y a su vez permiten a los agentes simplificar la invocación de acciones que generalmente son utilizadas juntas. Por ejemplo, un paquete de acciones podría contener acciones para abrir una aplicación, esperar a que esta se cargue y realizar una acción específica en la aplicación, etc.
- **Ejecución Segura:** El registro también permite a la MCE ejecutar acciones de forma segura, esto es el registro puede cambiar las funcionalidades de las acciones para que estas no se ejecuten directamente, en su lugar, pueden pedir una confirmación al usuario o bien ejecutar una acción simulada que no afecte al sistema. Esto es especialmente útil para acciones que pueden tener un impacto significativo en el sistema, como eliminar archivos, modificar configuraciones del sistema, etc.

6.5.2 Servicio Maestro-Esclavo

El servicio maestro-esclavo (SME) es el mecanismo encargado de gestionar las comunicaciones entre la máquina controlada (esclava) y la máquina que ejecuta la MCE (maestra).

Este servicio se realizó a través de una implementación basada en RabbitMQ con mensajes de tipo string entre ambas máquinas. Debido a la naturaleza de la MCE, se requiere que la ejecución de código en una máquina remota sea lo más segura posible, por lo que esta implementación ha sido reemplazada por un mecanismo externo basado en gRPC (Google Remote Procedure Call).

Esta implementación ha sido integrada dentro del registro de entidades, de forma que toda la carga de acciones, metadatos y ejecución de funcionalidades sea mediante este servicio.

6.5.3 Ejemplo de Funcionamiento de la MCE

De forma ilustrativa, el siguiente ejemplo muestra el funcionamiento de la MCE desde el punto de vista de la capa de núcleo. En este ejemplo, el usuario solicitará al agente que abra Spotify.

Nótese que múltiples mecanismos internos han sido omitidos para simplificar el ejemplo, a su vez, algunas simplificaciones han sido realizadas para facilitar la comprensión del funcionamiento de la MCE.

1. El usuario arranca la MCE:
 - (a) El registro de entidades es inicializado y se realiza una precarga de todas las acciones (y metadatos) disponibles en el espacio de acciones $\mathbb{B}^*$.
 - (b) El Interpretador, por ejemplo Pyxcel es inicializado y se enlaza con el registro de entidades y al mapa de correspondencia Λ .
 - (c) Se inicializa la capa de cognición y se carga el agente integrado en la MCE, por ejemplo, DarkVFR
 - (d) DarkVFR inicializa los módulos de la capa de cognición necesarios, como por ejemplo, inicializa el buffer de memoria y el estado cognitivo con los valores por defecto.

6.5. Capa de Núcleo

- (e) FastAP es inicializado y se enlaza con la capa de cognición. Crea un entorno virtual, analiza las dependencias del agente, y si existen dependencias obligatorias, carga en el registro de entidades las acciones necesarias.
 - (f) FastAP enlaza al agente (DarkVFR) con el interprete, permitiendole enviar acciones a la capa de ejecución.
 - (g) FastAP crea un servidor de control para el usuario, de forma que este pueda pararlo, cancelarlo o reiniciarlo en cualquier momento.
2. La MCE esta lista para recibir comandos, el usuario solicita al agente que abra Spotify usando el comando "Por favor abre Spotify".
- (a) FastAP recibe el mensaje del usuario y lo envía a DarkVFR
 - (b) DarkVFR consulta el registro de entidades y utiliza el autodescriptor de acciones para generar un set de instrucciones para el LLM.
 - (c) DarkVFR guarda el mensaje en su buffer de memoria, y llama al LLM para realizar una planificación.
 - (d) El LLM genera un plan de acción utilizando el lenguaje exelent, por ejemplo:

```
>> start opening spotify with priority normal
>>     sequentially run
>>         open_application "Spotify"
```
 - (e) DarkVFR envia el plan de acción a la capa de ejecución a traves del interprete.
3. El interprete recibe el plan de acción y lo envia a la capa de ejecución.
- (a) El interprete analiza el plan de acción y lo tokeniza, generando una estructura de datos que sigue el protocolo TSP.
 - (b) El interprete consulta el registro de entidades y encuentra la acción `open_application` y sus metadatos asociados a traves de su identificador único y el mapa de correspondencia Λ .
 - (c) El interprete crea un proceso paralelo y ejecuta la acción `open_application` con los argumentos necesarios, en este caso, el nombre de la aplicación "Spotify".
 - (d) El registro de entidades es disparado con la acción `open_application` y carga todas las librerias necesarias para la ejecución de la acción.

- (e) El interprete controla la ejecución de la acción y espera su resultado.
 - (f) Una vez que la acción ha sido completada, el interprete recibe el resultado y lo envía de vuelta a DarkVFR.
4. DarkVFR recibe el resultado de la acción y lo guarda en su buffer de memoria.
 5. DarkVFR decide finalizar la tarea debido a que el objetivo ha sido completado (omitido en este ejemplo).
 6. La ECM recibe el flag de tarea completada y finaliza la ejecución de la tarea.
 - (a) El registro de entidades revisa si hay librerías que pueden ser liberadas, y si es así, las elimina.
 - (b) FastAP deja el entorno virtual del agente bloqueado hasta próximo aviso.
 - (c) El interprete libera los recursos utilizados por acciones previas y revisa que no haya procesos abiertos.

6.6 Otros Componentes

A continuación se describen otros componentes de este proyecto que, si bien no son parte de la MCE, se han desarrollado para facilitar la experimentación y el desarrollo de la MCE.

6.6.1 Instalador Modular Automático

La MCE ha sido diseñada para ser modular y funcionar con distintos componentes de forma independiente. Por este motivo, se ha desarrollado un instalador que permite seleccionar distintas opciones de configuración y realizar una instalación automatizada de la MCE de forma unificada.

Este instalador permite, entre otros las siguientes configuraciones:

- Seleccionar la capa de ejecución a utilizar (Pyxcel o ROSA).
- Seleccionar el agente/s a utilizar (DarkVFR, FastReact, etc.) con sus respectivas dependencias.

- Detectar automáticamente el sistema operativo
- Seleccionar una instalación con soporte para el servicio maestro-esclavo
- Integrar claves y tokens de acceso a servicios externos que serán guardados en ficheros de configuración seguros y anónimos.
- Instalar la MCE en un entorno virtual de Conda (o directamente en el sistema)
- Actualizar automáticamente las variables de entorno del sistema necesarias para próximas ejecuciones.³
- Instalar herramientas de desarrollo como linters y formateadores de código.

Estas funcionalidades, a su vez, pueden ser accedidas desde tres interfaces distintas:

- **Interfaz Gráfica de Usuario:** Esta interfaz permite la instalación de la MCE utilizando la configuración más actualizada de forma sencilla y rápida.
- **Interfaz Gráfica de Desarrollador:** Esta interfaz permite la instalación de la MCE utilizando una configuración avanzada, permitiendo al usuario personalizar cada aspecto de la instalación y seleccionar componentes específicos según sus necesidades.
- **Script de Instalación:** Este script permite la instalación de la MCE de forma totalmente automática y sin intervención del usuario. Este script es especialmente útil para la integración en múltiples dispositivos con una configuración similar.

Este instalador ha sido desarrollado en Python y utiliza la librería `streamlit` para la creación de la interfaz gráfica. A su vez, el instalador está envuelto en dos scripts, uno para Windows y otro para Linux, que permiten la ejecución del instalador sin necesidad de instalar ninguna dependencia más allá de tener Python3 instalado en la máquina.

³En versiones previas del espacio de ejecución como el caso de ROSA, el instalador automáticamente realizaba el proceso de compilación y enlazado de los paquetes de ROS2 necesarios.

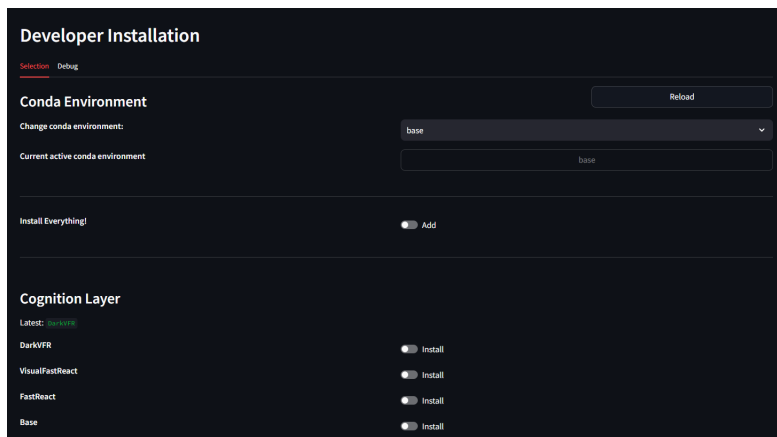


Figure 6.3: Interfaz gráfica de desarrollador. Fuente: elaboración propia.

6.6.2 Imagen y Kernel para Sistemas Embebidos

Con el fin de poder integrar la MCE en un sistema embebido, (donde la MCE se ejecuta como maestro en el sistema embebido y la máquina esclava es un dispositivo externo) se ha desarrollado una imagen de sistema y un bootloader personalizado.

Ambas imagenes han sido desarrolladas desde la placa de desarrollo IMX8MP de NXP, con un microprocesador ARM Cortex A53 de 4 núcleos. No se asegura que estas imagenes sean compatibles en su totalidad con otros sistemas embebidos, sin embargo, se ha realizado un esfuerzo para que esta imagen sea lo más portable posible y pueda ser utilizada en otros entornos.

Kernel Personalizado

Usando el repositorio *linux-imx* de NXP (nd) como base. Se ha compilado un kernel personalizado que contiene los requisitos mínimos necesarios para la ejecución de la MCE.

Concretamente, en este kernel se han incluido los drivers necesarios para la comunicación con la máquina esclava a través de MarvellWifi; arquitectura de red estandar para comunicaciones a traves de, por ejemplo, el chipset 88W8997 de Marvell.

A su vez, se han generado los headers necesarios para la compilación de drivers

adicionales que puedan ser necesarios en el futuro.

Junto a este kernel, se ha generado un DTB (Device Tree Blob) que describe la configuración del hardware del sistema embebido y permite que el kernel reconozca y utilice correctamente los dispositivos conectados.

Finalmente, se ha generado fichero `extlinux.conf` que permite configurar el arranque del sistema y seleccionar el root filesystem a utilizar.

Imagen del Sistema

La imagen del sistema ha sido generada a partir de la distribución *Arch Linux ARM* y contiene los siguientes componentes ya instalados:

- **Python 3:** El lenguaje de programación utilizado para la MCE, así como todas las dependencias necesarias para su ejecución.
- **Drivers de Red:** Los drivers necesarios para la comunicación con la máquina esclava a través de MarvellWifi.
- **Herramientas de Desarrollo:** Herramientas como `git`, `make`, `gcc` y `bttop` para la gestión del sistema y desarrollo de la MCE.
- **Arranque Automático:** Se ha configurado el sistema para que la MCE se pueda arrancar automáticamente al inicio del sistema gracias a un servicio de `systemd`.
- **Configuración básica de Seguridad:** Se ha configurado el sistema para que la MCE se ejecute con un usuario específico y se han deshabilitado los servicios innecesarios para minimizar los riesgos de posibles ataques.

Nótese como a esta imagen no se le ha integrado una interfaz gráfica o gestor de ventanas, pues esta solo es necesaria en la máquina esclava, donde se ejecutarán las acciones del espacio de acciones $\mathbb{B}^*$.

Integración de Kernel e Imagen del Sistema

Para integrar el kernel y la imagen del sistema, basta con crear dos particiones en la tarjeta SD del sistema embebido:

- **Partición de arranque:** Esta partición contendrá la imagen del kernel personalizado, el DTB y el fichero `extlinux.conf`.
- **Partición raíz:** Esta partición contendrá la imagen del sistema operativo.

Es importante revisar que el bootloader este configurado para arrancar el kernel desde la partición de arranque. Este bootloader generalmente se encuentra en el comienzo de la memoria de la tarjeta SD y esta disponible en la documentación del sistema embebido.

6.6.3 Integración Continua

Utilizando las librerías `pre-commit` y `pytest` se ha generado un set de pruebas automatizadas que permite verificar el correcto funcionamiento de la MCE y sus componentes.

Estas pruebas son ejecutadas automáticamente cada vez que se realiza un commit en el repositorio de la MCE, permitiendo verificar que los cambios realizados no rompen ninguna funcionalidad existente.

Entre los tests automáticos se encuentran:

- **Test Unitarios:** Estos tests verifican el correcto funcionamiento de mecanismos independientes como el registro de entidades, el interprete, etc.
- **Test de Integración:** Estos tests verifican que los distintos componentes de la MCE funcionan correctamente juntos. Por ejemplo, se verifica que el interprete puede cargar acciones desde el registro de entidades y ejecutarlas
- **Bootstraps:** Estos tests pueden ser ejecutados manualmente y verifican que la MCE puede ser arrancada correctamente, utilizando distintos componentes y configuraciones.
- **Tests del espacio de acciones $\mathbb{B}^*$:** Estos tests verifican que las acciones del espacio de acciones son realmente ejecutadas en la máquina local y/o máquina esclava y que modifican el entorno de forma correcta.

Adicionalmente, se ha configurado el sistema de pre-commit para que realice un proceso de formateo y revisión de código antes de realizar un commit. Esto permite mantener un código limpio y legible, así como evitar errores comunes de sintaxis o estilo.

7. EXPERIMENTACIÓN

En el capítulo anterior, se presentó de forma general los componentes más relevantes de la MCE, describiendo entre otros, los mecanismos para generación de tokens en Λ , el mecanismo de traducción de dichos tokens a acciones en el espacio de acciones $\mathbb{B}^*$ y su integración con la capa de cognición de la MCE.

Sin embargo, aun no se han presentado los agentes que utilizan esta arquitectura para resolver las tareas propuestas por el usuario. Formalmente, aun no se ha definido la función $T_C(p, C^*) = \lambda$.

Debido a la naturaleza experimental de dicha función, se ha decidido implementar 8 distintas aproximación agénticas para este problema, cada una con un enfoque diferente, pero todas compatibles y utilizando los mecanismos de la MCE.

Nótese que dada la arquitectura propuesta todos los agentes utilizan el protocolo FastAP para su comunicación con la MCE, lo que los hace independientes del resto de componentes.

A continuación se describen los agentes implementados, sus características principales y sus diferencias con el resto de agentes.

7.1 Aproximaciones Agénticas

A continuación se describen las aproximaciones agénticas implementadas en la MCE. Es importante denotar que algunos de estos agentes son variantes de otros, pero con un enfoque diferente, por lo que se han decidido presentar de forma separada:

- **Planex:** Basado en el juego de palabras referente a "Planificador Exelent", este es un agente que utiliza un mecanismo de razonamiento y planificación definido en 3 fases.

La primera fase genera un plan de acción a largo plazo utilizando acciones perceptivas que le permiten observar el entorno; la segunda fase consiste en

una reducción del plan generado en un conjunto de acciones que resuelven dicha tarea usando acciones dentro del espacio $\mathbb{B}^*$; la tercera fase consiste en la generación de lenguaje exelente para enviarlo posteriormente a la capa de Ejecución de la MCE.

Este agente es integrado dentro de un bucle de control que mantiene al agente en un ciclo de observación, planificación y ejecución hasta que la tarea sea resuelta o el agente decida detenerse.

- **PlanexV2:** Esta variante de Planex que introduce una fase adicional de recuperación. En este sentido, cuando el agente genera halucinaciones o errores son recibidos de la capa de ejecución, el agente entra en un bucle de recuperación que le permite observar sus propios mensajes, los errores obtenidos y reajustar alguna de las 3 fases de Planex para intentar resolver la tarea de forma exitosa.

Este mecanismo permite al agente recuperarse de forma más ágil, sin tener que realizar una iteración o reinicio completo para poder reaccionar ante errores.

- **RePlan:** Este agente es una variante de Planex que utiliza una filosofía de planificación basada en *ReAct*, una de las metodologías de ingeniería de prompting discutida en la sección 2.4. Así pues, *ReAct* utiliza un ciclo de razonamiento y acción, donde el agente genera un razonamiento sobre la tarea a resolver, seguido de una acción que le permite observar y/o modificar su entorno.

Manteniendo este ciclo de razonamiento y acción, *RePlan* puede utilizar las capacidades de Planex de forma dinámica, decidiendo cuando activar cada fase de Planex de forma autónoma. Con esto, *RePlan* puede decidir si generar un plan de acción a largo plazo, o generar lenguaje exelent para enviarlo a la capa de ejecución.

- **Xplore:** *Xplore* es un agente que utiliza un enfoque de exploración y explotación para resolver las tareas propuestas por el usuario. Para ello, *Xplore* utiliza otra técnica de ingeniería de prompting llamada *Grafos Agéntico*. Un grafo agéntico consiste en un grafo de acciones y estados, donde cada nodo representa un comportamiento del agente, y cada arista representa una transición condicional entre comportamientos.

El grafo agéntico diseñado contempla los estados: planificación a largo plazo, planificación a corto plazo, recuperación, aprendizaje y refinamiento. Cada uno de estos estados permite representar la información obtenida del

entorno en distintos formatos, que permitirán a Xplore adaptarse a su entorno y no solo interactuar de forma reactiva, sino también modificar su comportamiento a largo plazo a medida que su entorno cambia.

- **FastReact:** FastReact es un agente que utiliza un mecanismo de razonamiento y acción basado en ReAct, pero con un set de optimizaciones que permiten al agente razonar y actuar de forma paralela y mucho más eficiente.

Este agente ya no utiliza bucles de control, ni está conectado con Planex, sino que directamente es capaz de interactuar con su entorno usando puramente lenguaje exelent.

- **Visual-FastReact:** Este agente, abreviado como VFR es una variante de FastReact que integra capacidades de visión para obtener no solo información desde el espacio de acciones, sino también desde imágenes o capturas de pantalla del entorno.

De esta forma, VFR puede observar su entorno de forma más completa, obteniendo información visual que resulta imposible de obtener únicamente con el espacio de acciones $\mathbb{B}^*$.

- **MinimalVFR:** MinimalVFR es una variante de Visual-FastReact que simplifica su arquitectura manteniendo el núcleo del ciclo de razonamiento y acción.

A nivel técnico, el agente prescinde de herramientas de flujo como Lang-Graph y basa su ejecución en una arquitectura modular, con métodos claramente separados y reemplazables que pueden ser sobrescritos para extender su comportamiento.

Esta unificación del ciclo cognitivo permite reducir la complejidad y aumentar la flexibilidad del agente, facilitando el desarrollo de nuevas variantes y mejorando la adaptabilidad a distintos entornos.

- **DarkVFR:** DarkVFR es una variante de MinimalVFR que optimiza el proceso de extracción de información visual, eliminando información irrelevante y enfocándose únicamente en la información que está relacionada con la tarea a resolver.

A su vez, DarkVFR integra meta-acciones para controlar su comportamiento, así como un set de optimizaciones a nivel de memoria que le permiten eliminar información textual que puede no ser conveniente para la tarea a resolver.

7.2. Características Técnicas y Funcionales

Estos agentes han sido implementados utilizando los mecanismos de ingeniería de prompting presentados en la sección 2.4, así como mecanismos de control de mensajes, llamada a funciones o estructuración de prompts presentados en la sección 2.6.

Para el desarrollo de los agentes, se ha utilizado la librería Langchain y Langgraph, así como módulos customizados para cada agente escritos totalmente en Python.

7.2 Características Técnicas y Funcionales

A continuación, se presentan las características técnicas y funcionales de cada agente, seguidas de un análisis comparativo que permitirá formular hipótesis sobre su comportamiento esperado en distintos escenarios.

Table 7.1: Tipo de Ciclo Cognitivo

Agente	Ciclo Cognitivo Principal
Planex	Observación, planificación y ejecución
PlanexV2	Observación, planificación, ejecución + recuperación
RePlan	ReAct + Planex
Xplore	Grafo de comportamientos
FastReact	ReAct optimizado
VFR	ReAct optimizado + capacidades visuales
MinimalVFR	React optimizado + capacidades visuales
DarkVFR	React optimizado + capacidades visuales eficientes

Esta tabla presenta la clasificación de los agentes en función del tipo de ciclo cognitivo que estructura su comportamiento. Esta característica define la forma en que el agente percibe, planifica y actúa sobre el entorno.

Se identifican tres corrientes principales:

- **Ciclo predeterminado (Planex, PlanexV2):** Estos agentes siguen un flujo fijo de observación, planificación y ejecución. Aunque este enfoque facilita el control y la predicción del comportamiento, también lo hace vulnerable a errores no previstos en fases tempranas.

7.2. Características Técnicas y Funcionales

- **Grafo de comportamientos (Xplore):** Representa una estructura más dinámica, en la que el agente transiciona entre diferentes comportamientos según condiciones del entorno. Esto le permite adaptarse mejor a cambios y aprender patrones de interacción, aunque requiere un diseño que contemple de forma efectiva los estados más representativos de la tarea a resolver.
- **Razonamiento tipo ReAct (RePlan, FastReact, etc.):** Estos agentes generan decisiones a través de ciclos de razonamiento y acción. Son altamente adaptativos y no requieren un flujo predefinido, aunque pueden ser menos predecibles y más sensibles a errores de generación.

Table 7.2: Modularidad y Capacidades Internas de los Agentes

Agente	Modularidad	Visión	Recuperación	Memoria	Aprendizaje	Meta-acciones
Planex	Media	—	—	—	—	—
PlanexV2	Media	—	✓	—	—	—
RePlan	Alta	—	✓	✓	—	—
Xplore	Media	✓	✓	✓	✓	—
FastReact	Alta	—	✓	✓	—	—
VFR	Alta	✓	✓	✓	—	—
MinimalVFR	Muy alta	✓	✓	✓	—	—
DarkVFR	Muy alta	✓	✓	✓	—	✓

Esta nueva tabla complementa la anterior describiendo aspectos internos clave que determinan el grado de autonomía, adaptabilidad y extensibilidad de cada agente. Estas características permiten entender no solo cómo están contruidos, sino qué tipo de comportamiento podríamos esperar cuando se enfrentan a tareas reales.

- **Modularidad:** La evolución de los agentes refleja un diseño cada vez más flexible. Mientras Planex y Xplore conservan una modularidad media, agentes como MinimalVFR y DarkVFR alcanzan una arquitectura altamente desacoplada, facilitando su extensión y mantenimiento.
- **Visión:** Solo Xplore y los agentes de la familia VFR incorporan percepción visual, lo que les otorga una ventaja clara en tareas que requieren interpretar información del entorno más allá del canal textual.

7.3. Hipótesis del Experimento

- **Recuperación:** Excepto Planex, todos los agentes incorporan algún tipo de mecanismo para recuperarse de errores. Esto sugiere que, en escenarios no deterministas o propensos a fallos, la mayoría de agentes tendrá al menos una vía de corrección.
- **Memoria:** Se observa un aumento progresivo en la capacidad de memoria. Los agentes más avanzados no solo conservan información entre pasos, sino que la reutilizan y modifican activamente para razonar o tomar decisiones futuras.
- **Aprendizaje:** Xplore destaca por incorporar mecanismos de aprendizaje. Esto lo posiciona como un candidato adecuado para tareas iterativas o con entornos parcialmente observables, donde la adaptación continua es una ventaja competitiva.
- **Meta-acciones:** Únicamente DarkVFR tiene la capacidad de autorregular su comportamiento mediante meta-acciones. Al igual que con Xplore, esto le supone una ventaja competitiva en tareas donde la adaptabilidad y la auto-optimización son cruciales.

En conjunto, la tabla muestra una evolución clara desde agentes básicos y estructurados hacia otros más modulares, perceptivos y adaptativos. Esta progresión plantea una pregunta central que será abordada en los siguientes experimentos:

¿Realmente estas capacidades avanzadas se traducen en un mejor rendimiento en tareas prácticas?

7.3 Hipótesis del Experimento

En las posteriores secciones, se presentarán los entornos de pruebas diseñados para evaluar el rendimiento de los agentes, así como las métricas de rendimiento que se utilizarán para medir su eficacia.

La hipótesis central que se plantea con estos experimentos es que los agentes desarrollados no solo son capaces de operar de forma autónoma, sino que además logran integrarse correctamente con la MCE siguiendo la función de cognición definida en el modelo teórico. En este sentido, se espera que la interacción entre los agentes dentro de la MCE sea equivalente a la función $T_C(p, C^*) = \lambda$, donde p

es el problema a resolver que se planteara en el entorno de pruebas, C^* es equivalente al agente y λ es el lenguaje excelente generado por los agentes para resolver dicho problema.

Con esto, el objetivo del experimento no se limita a medir el rendimiento individual de cada agente, sino también a evaluar la validez del modelo teórico global, entendido como la interacción coordinada de cada uno de los componentes dentro de un sistema cognitivo funcional. Las métricas de rendimiento obtenidas en estos entornos permitirán valorar tanto la eficacia de cada agente como la viabilidad práctica de la MCE como arquitectura agéntica integrada.

7.4 Entorno de Pruebas y Evaluación

Para evaluar el rendimiento de los agentes, se han diseñado 3 entornos de pruebas que integrarán a cada agente dentro de la MCE. A su vez, este será desplegado de forma local en un ordenador convencional con un sistema operativo Windows 11, sin acceso a la GPU, con acceso a internet y con configuraciones y programas aleatorios, para simular un entorno realista y no controlado.

Los objetivos planteados en cada uno de estos entornos son los siguientes:

1. **Hello World:** Este entorno de pruebas cuenta con un editor de texto abierto y preparado para recibir entradas de texto. El objetivo del agente es escribir el texto "Hola Mundo" en dicho editor, Con esta tarea se pretende evaluar la capacidad del agente para interactuar con su entorno utilizando el espacio de acciones.

La métrica de rendimiento más relevante será en este caso será el tiempo que tarda el agente en completar la tarea, tratándose esta de una tarea sencilla que puede ser directamente resuelta con una única acción.

Para realizar esta tarea, el agente deberá ser capaz de realizar todas las acciones necesarias, habiendo recibido como entrada el siguiente texto:

"Por favor, escribe 'Hola Mundo'"

2. **Play a Song:** En este entorno, el agente tendrá Spotify abierto y preparado con una canción aleatoria. Sin embargo, esta aplicación se encontrará minimizada, por lo que el agente deberá encontrar la aplicación, maximizarla y reproducir la canción.

7.4. Entorno de Pruebas y Evaluación

Es importante denotar que esta tarea no requiere de forma obligatoria de capacidades de visión, ni de control de mouse, pues el agente puede descubrir que presionando la tecla Espacio se reproduce la canción, y que la aplicación es accesible desde la barra de tareas de Windows.

Con esta tarea se pretende evaluar la capacidad del agente no solo para interactuar con su entorno, sino también para analizar el estado del mismo y descubrir acciones que le permitan resolver la tarea de forma efectiva. En este caso, la métrica de rendimiento no solo será el tiempo que tarda el agente en completar la tarea, sino también el número de acciones que ha realizado para resolverla, así como si ha sido capaz de resolver la tarea. No se evaluará si se han utilizado o no capacidades de visión.

La entrada que recibirá el agente para esta prueba será:

"Reproduce una canción en Spotify"

3. **Buy on Amazon:** En este entorno el agente iniciará en el escritorio de Windows. Tendrá aplicaciones abiertas como Spotify o Visual Studio Code, pero no tendrá ningún navegador abierto. El objetivo del agente será: descubrir que navegador tiene instalado, abrirlo, encontrar una página web de Amazon que tenga un producto especificado y o bien añadirlo al carrito de la compra, o bien comprarlo directamente con el botón de "Compra en un click"

Las métricas de rendimiento más relevantes en este caso serán la eficiencia del agente para realizar la tarea, donde dicha eficiencia se medirá tanto en el número de acciones como en la cantidad de tokens utilizados (en el proceso de razonamiento dentro de la LLM) para completar la tarea.

Esta tarea es mucho más compleja que las anteriores, pues requiere de capacidades de razonamiento, planificación y descubrimiento de acciones de forma continuada, dinámica y efectiva. A su vez, el agente deberá ser capaz de interactuar con el navegador y con la página web de Amazon, lo que implica que deberá ser capaz de interpretar el contenido de la página, identificar los elementos relevantes y realizar acciones sobre ellos, como hacer clic en botones o rellenar formularios.

La entrada que recibirá el agente para esta prueba será:

"Busca un portátil en Amazon y compralo o añádelo al carrito."

Notese que como medida de seguridad, los agentes contarán con un límite de tiempo y acciones para completar cada una de estas tareas. Estas restricciones

7.4. Entorno de Pruebas y Evaluación

están diseñadas para evitar que los agentes se queden atrapados en bucles infinitos o realicen acciones no deseadas.

El tiempo máximo para completar las tareas será de 2.5 minutos para la prueba de *Hello World*, 5 minutos para la prueba de *Play a Song* y 10 minutos para la prueba de *Buy on Amazon*.

La cantidad máxima de acciones que podrá realizar el agente será de 3 acciones para la prueba de *Hello World*, 10 acciones para la prueba de *Play a Song* y 50 acciones para la prueba de *Buy on Amazon*.

Debido a la naturaleza de cada agente, las acciones que requieran de capacidades de visión y/o meta-acciones solo estarán disponibles para aquellos agentes que sean compatibles con ellas, mientras que el resto de agentes no tendrán dichas acciones dentro de su espacio de acciones $\mathbb{B}^*$.

BIBLIOGRAPHY

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023). Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Aeronautiques, C., Howe, A., Knoblock, C., McDermott, I. D., Ram, A., Veloso, M., Weld, D., Sri, D. W., Barrett, A., Christianson, D., et al. (1998). Pddl— the planning domain definition language. *Technical Report, Tech. Rep.*
- AI-Engineer-Foundation (2023). Agentprotocol.
- Fezari, M. and Ali-Al-Dahoud, A. A.-D. (2023). From gpt to autogpt: a brief attention in nlp processing using dl. *Preprint*.
- Fikes, R. E. and Nilsson, N. J. (1971). Strips: A new approach to the application of theorem proving to problem solving. *Artificial intelligence*, 2(3-4):189–208.
- Laird, J. E. (2019). *The Soar cognitive architecture*. MIT press.
- LangChain (2024). Langchain documentation: Agents module. Accessed: 2024-07-05.
- McCarthy, J., Minsky, M. L., Rochester, N., and Shannon, C. E. (2006). A proposal for the dartmouth summer research project on artificial intelligence, august 31, 1955. *AI magazine*, 27(4):12–12.
- Morris, M. R., Sohl-Dickstein, J., Fiedel, N., Warkentin, T., Dafoe, A., Faust, A., Farabet, C., and Legg, S. (2023). Levels of agi: Operationalizing progress on the path to agi. *arXiv preprint arXiv:2311.02462*.
- Newell, A., Shaw, J. C., and Simon, H. A. (1959). Report on a general problem solving program. In *IFIP congress*, volume 256, page 64. Pittsburgh, PA.
- Norris Jr, D., Moran, B., Scudder, J., and Quinones, D. (1978). A computer simulation of the tension test. *Journal of the Mechanics and Physics of Solids*, 26(1):1–19.
- NXP (n.d.). linux-imx. <https://github.com/nxp-imx/linux-imx>. Github Repository.

- Reworkd (2023). Agentgpt.
- Russell, S. J. and Norvig, P. (2016). *Artificial intelligence: a modern approach*. Pearson.
- Saeidnia, H. R. (2023). Welcome to the gemini era: Google deepmind and the information industry. *Library Hi Tech News*, (ahead-of-print).
- Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., and Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*.
- Searle, J. R. (1980). Minds, brains, and programs. *Behavioral and brain sciences*, 3(3):417–424.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- TransformerOptimus (2023). Superagi.
- Vervaeke, J. A. (1999). *The naturalistic imperative in cognitive science*. PhD thesis.
- Viteri, S., Nanda, N., and Smith, J. (2024). Scaling monosemanticity in transformer circuits. Accessed: 2024-07-08.
- Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.
- Wu, A. (2023). Metagpt.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. (2023). Autogen: Enabling next-gen llm applications via multi-agent conversation framework.